# Architektura systemu TestGEN

## Spis treści


- Architektura — indeks
  - Przegląd
  - Decyzje architektoniczne (ADR)
  - API
  - Architektury obszarów
  - Powiązane
- Dokumentacja architektury TestGEN — C4 i diagramy
  - Spis treści
  - 1. Wprowadzenie
  - 2. C4 Level 1 — Diagram kontekstowy (Context)
  - 3. C4 Level 2 — Diagram kontenerów (Container)
  - 4. C4 Level 3 — Diagramy komponentów (Component)
  - 5. C4 Level 4 — Diagramy kodu (Code)
  - 6. Diagramy przepływu danych
  - 7. Diagramy wdrożeniowe (Deployment)
  - 8. Diagramy sekwencji
  - 9. Decyzje architektoniczne (ADR)
  - 10. Konwencje diagramów
  - Powiązane dokumenty
- Architektura — TestGEN (P1)
  - C4 — Context
  - C4 — Container
  - Warstwy backendu
  - Przepływ asynchroniczny (przykład: analiza dokumentacji)
  - Izolacja danych (RLS)
  - Powiązane
- Konwencje kodu — TestGEN (P1)
  - Nazewnictwo
  - Struktura katalogów
  - Testy
  - Git flow
  - Commity
  - Linting / formatowanie
  - Code review
- Workflow dla deweloperów — TestGEN (P1)
  - Dodanie nowego języka parsera (AST)
  - Dodanie nowego frameworku testowego

- Dodanie nowego formatu dokumentacji
- Dodanie nowej reguły analizy dokumentacji
- Dodanie nowej tabeli (z RLS)
- Uruchomienie zadania asynchronicznego lokalnie
- Aktualizacja golden data przy celowej zmianie

ADR-001: Custom monorepo (FastAPI + Angular + Flutter)
- Status
- Kontekst
- Decyzja
- Konsekwencje
- Powiązane

ADR-002: Multi-tenancy z Row-Level Security (RLS)
- Status
- Kontekst
- Decyzja
- Konsekwencje
- Powiązane

ADR-003: Argon2id + RS256 JWT (asymmetric)
- Status
- Kontekst
- Decyzja
- Konsekwencje
- Powiązane

ADR-004: Celery + RabbitMQ (przetwarzanie asynchroniczne)
- Status
- Kontekst
- Decyzja
- Konsekwencje
- Powiązane

ADR-005: tree-sitter jako parser AST
- Status
- Kontekst
- Decyzja
- Konsekwencje
- Powiązane

ADR-006: Jinja2 jako silnik szablonów kodu testowego
- Status
- Kontekst
- Decyzja
- Konsekwencje
- Powiązane

ADR-007: Wsparcie wieloframeworkowe dla testów E2E
- Status
- Kontekst
- Decyzja
- Konsekwencje

- Powiązane
- ADR-008: Golden data do walidacji analiz
  - Status
  - Kontekst
  - Decyzja
  - Konsekwencje
  - Powiązane
- ADR-009: Przepływ przeglądu wygenerowanych elementów
  - Status
  - Kontekst
  - Decyzja
  - Konsekwencje
  - Powiązane
- ADR-010: Logowanie audytowe operacji
  - Status
  - Kontekst
  - Decyzja
  - Konsekwencje
  - Powiązane
- ADR-011: UML wizualne Tier 2 (SVG/PDF/raster) + adapter visual→structural
  - Status
  - Kontekst
  - Decyzja
  - Konsekwencje
  - Powiązane
- ADR-012: SikuliX image-based — dedykowany pipeline PNG + `pattern_images`
  - Status
  - Kontekst
  - Decyzja
  - Konsekwencje
  - Powiązane
- ADR-013: Traceability — semantyka Gherkin-proxy + provenance `source` / `confidence`
  - Status
  - Kontekst
  - Decyzja
  - Konsekwencje
  - Powiązane
- ADR-NNN: Tytuł decyzji
  - Status
  - Kontekst
  - Decyzja
  - Konsekwencje
  - Powiązane
- Przykłady API — Upload i analiza kodu (AST)
  - Upload kodu źródłowego

- Uruchomienie analizy (async)
- Status i wyniki
- Python (httpx)
- Typowe błędy

Przykłady API — Logi audytowe (admin)
- Charakterystyka logów
- Co jest logowane
- Dostęp (rola admin)
- Przykładowy kształt wpisu
- Powiązane

Przykłady API — Autentykacja i MFA
- Rejestracja
- Logowanie
- Wybór tenanta
- Odświeżanie i wylogowanie
- MFA (TOTP, opcjonalne)
- Typowe błędy

Przykłady API — Analiza dokumentacji (Epic 2)
- 1. Upload (async) — przez `/projects/{id}/upload-documents`
- 2. Status analizy
- 3. Pełny wynik analizy
- 4. Wymagania + workflow
- 5. Klasyfikacja, sekcje, glosariusz, niespójności
- 6. Ponowna analiza (reparse)
- Moduły zależne (tylko do odczytu)
- Typowe błędy

Przykłady API — Projekty i upload dokumentacji
- Utworzenie projektu
- Lista i pobranie
- Aktualizacja i usunięcie
- Preprocessing kodu (async)
- Upload dokumentacji (Epic 2, async)
- Lista plików projektu
- Python (httpx)
- Typowe błędy

Przykłady API — Przypadki testowe i przegląd
- Utworzenie przypadku testowego
- Lista i pobranie
- Edycja i usunięcie
- Przegląd (akceptacja)
- Wykonanie (async)
- Python (httpx)
- Typowe błędy

Przykłady API — moduł UML/GUI (Epic 3)
- Autoryzacja
- Kody statusu

- UML (Stories 3.1–3.7)
- GUI (Stories 3.8–3.10)
- Gherkin / BDD (Story 3.11)
- Codegen (Stories 3.12, 3.13)
- Traceability (Story 3.15)
- Sekwencje wywołań
- Powiązania ze Stories

Przegląd API — platforma TestGEN (P1)
- Konwencje
- Mapa domen (P1 = Epic 1 + Epic 2)

Architektura modułu billingowego — przegląd
1. Cel modułu
2. Diagram przepływów
3. Moduły i ich odpowiedzialności
4. Zadania okresowe (Celery beat)
5. Konfiguracja
6. Zasady projektowe
7. Powiązane dokumenty

Plany subskrypcji i limity
1. Model 3-planowy
2. Macierz limitów
3. Progi modułów analizy (feature gating)
4. Seed planów
5. Migracja legacy (043)

Integracja tpay — checkout
1. Decyzje architektoniczne
2. Klient Open API
3. Przepływ checkoutu (Story 7.4)
4. Strony powrotu i status transakcji
5. Środowisko sandbox

Webhooki tpay
1. Endpoint i autoryzacja
2. Macierz typów notyfikacji
3. Semantyka odpowiedzi
4. Efekty transakcji (mark_paid)
5. Monitoring (Story 7.3 AC6)
6. Replay (Story 7.3 AC7)

Tokeny płatności, odnowienia i dunning
1. Model PaymentToken
2. Rejestracja karty bez obciążenia (Story 7.2 AC2)
3. Aliasy BLIK (Płatności Powtarzalne, model A)
4. Odnowienia cykliczne — scheduler 03:30
5. Polityka dunningu

Zmiany planu i proration
1. Klasyfikacja zmiany
2. Formuła proration

3. Przepływ upgrade
4. Przepływ downgrade (zaplanowany)
5. Historia zmian
6. Interakcje z anulowaniem subskrypcji (Story 7.7)

Limity i zużycie zasobów

1. Liczniki i egzekwowanie
2. Dashboard zużycia (Story 7.9)
3. Prognoza limitu
4. Historia zużycia

Powiadomienia o limitach i feature gating

1. Powiadomienia o progach zużycia (Story 7.11)
2. Feature gating modułów analizy (Story 7.13, FR71)

Faktury VAT

1. Cykl życia faktury
2. Numeracja — `FV/{rok}/{miesiąc}/{sekwencja}`
3. PDF, storage, wysyłka
4. Dane fakturowe tenanta (AC4)
5. API faktur

Modele danych modułu billingowego (ERD)

1. Diagram relacji
2. Tabele
3. Rejestr migracji

Architektura frontendu — przegląd

1. Cel
2. Stack technologiczny
3. Routing
4. Komunikacja z backendem
5. Dostępność jako wymiar poprzeczny
6. Powiązane dokumenty

Struktura aplikacji Angular

1. Warstwy
2. Shared — komponenty
3. Shared — dyrektywy (dostępność)
4. Shared — serwisy
5. Style i tokeny
6. Core — guards i interceptors
7. Konwencje testowe

Mapa ekranów i przepływów

1. Dashboard (Story 6-1)
2. Upload (Story 6-2)
3. Konfiguracja i uruchamianie analiz (Story 6-3)
4. Przeglądarka wyników (Story 6-4)
5. Zarządzanie test case'ami (Story 6-5)
6. Raporty (Stories 6-6, 6-7 — FR43/FR44/FR45)
7. Ekran błędów i statusów
8. Typowy przepływ użytkownika

Fundamenty dostępności — WCAG 2.1 AA
1. Zasada
2. Warstwa stylów ( `shared/styles/` )
3. Semantyka HTML
4. Klawiatura
5. Komunikaty asynchroniczne
6. Formularze
7. Weryfikacja

Motywy dostępności — wysoki kontrast i redukcja animacji
1. Mechanizm
2. Pasek dostępności (toolbar)
3. Tryb wysokiego kontrastu
4. Redukcja animacji
5. Zoom i responsywność

ARIA i czytniki ekranu
1. Zasady użycia ARIA
2. Nawigacja czytnikiem
3. Komunikaty asynchroniczne (aria-live)
4. Interakcje klawiaturowe widgetów
5. Polonizacja komponentów Material
6. Weryfikacja z czytnikami

Powłoka Flutter WebView (mobile/desktop)
1. Koncepcja
2. Platformy
3. Struktura ( `mobile/lib/` )
4. Mostek JS ( `js_bridge.dart` ↔ `flutter-bridge.service.ts` )
5. Deep linki i powiadomienia
6. Tryb offline
7. Konfiguracja i budowa

Strategia testowa frontendu
1. Poziomy testów
2. Testy komponentowe (Vitest)
3. E2E — Playwright
4. Audyt dostępności w CI (axe-core)
5. Produkcja (AOT)

# Architektura — indeks

> Dokumentacja techniczna platformy. Wzorce, decyzje (ADR), API, architektury UI i modułu billingowego.

## Przegląd

- architecture.md — **kompletna dokumentacja architektury C4:** diagramy kontekstowe, kontenerów, komponentów (M1-M6), kodu (dual-path, pipeline, parsery), przepływy danych, wdrożenia (Docker Compose / K8s), sekwencje + 20 diagramów PlantUML
- system.md — skrócony opis architektury (ASCII art)
- conventions.md — konwencje repo/kodu/dokumentacji
- workflows.md — przepływy pracy (dev → review → release)
- diagrams/ — pliki źródłowe PlantUML ( `.puml` ) — C4 L1-L4, przepływy, wdrożenia, sekwencje

## Decyzje architektoniczne (ADR)

adr/ — 13 decyzji + szablon: monorepo (001), multi-tenancy RLS (002), Argon2id+RS256 (003), Celery+RabbitMQ (004), tree-sitter (005), Jinja2 (006), frameworki E2E (007), golden data (008), code review (009), audit log (010), UML Tier 2 (011), SikuliX (012), traceability proxy (013).

## API

- api/overview.md — przegląd API ( `/api/v1` )
- api/examples/ — przykłady żądań/odpowiedzi per obszar: auth, projects, analysis, test cases, documents, UML/GUI, audit

Dokumentacja żywa: platformowe `/docs` (OpenAPI 3.1 generowane z kodu). Kompletne pokrycie API (Epic 8) — story 8-5/8-13, uzupełni ten katalog.

## Architektury obszarów

| Obszar | Dokumenty |
|---|---|
| Interfejs webowy | interfejs-webowy/ — stack Angular 21, struktura core/shared/features, ekrany i przepływy, dostępność (WCAG fundamenty/motywy/ARIA), shell Flutter, strategia testowa |
| Billing | billing/ — plany i limity, integracja tpay (checkout, webhooki, tokeny), zmiany planu i proration, limity/zużycie, powiadomienia i gating, faktury VAT, modele danych |

## Powiązane

- Szczegóły modułów analiz → ../ai-modules/README.md
- Wdrożenia/środowisko → ../deployment-guide/README.md

# Dokumentacja architektury TestGEN — C4 i diagramy

> **Wersja:** 1.0 · **Ostatnia aktualizacja:** 2026-09 · **Grupa docelowa:** Architekci, deweloperzy, DevOps

---

## Spis treści


1. Wprowadzenie
2. C4 Level 1 — Diagram kontekstowy (Context)
3. C4 Level 2 — Diagram kontenerów (Container)
4. C4 Level 3 — Diagramy komponentów (Component)
5. C4 Level 4 — Diagramy kodu (Code)
6. Diagramy przepływu danych
7. Diagramy wdrożeniowe (Deployment)
8. Diagramy sekwencji
9. Decyzje architektoniczne (ADR)
10. Konwencje diagramów


---

## 1. Wprowadzenie

### 1.1. Cel dokumentu

Dokument przedstawia architekturę platformy TestGEN na czterech poziomach szczegółowości zgodnie z modelem C4 (Context, Container, Component, Code). Uzupełniają go diagramy przepływu danych, diagramy wdrożeniowe oraz diagramy sekwencji kluczowych przepływów.

### 1.2. Czym jest TestGEN

TestGEN to platforma SaaS B2B do automatycznego generowania przypadków testowych i kodu testowego z wykorzystaniem sztucznej inteligencji. System analizuje kod źródłowy, dokumentację, diagramy UML i zrzuty GUI, a następnie generuje kompletne zestawy testów z priorytetyzacją i kodem gotowym do uruchomienia.

### 1.3. Stos technologiczny

| Warstwa | Technologia | Wersja |
|---|---|---|
| Frontend (SPA) | Angular, Material 3, Standalone Components | 21 |
| Frontend (mobile/desktop) | Flutter WebView | 3.x |

| Warstwa | Technologia | Wersja |
|---|---|---|
| Backend (API) | Python, FastAPI, SQLAlchemy 2.0 async | 3.12 |
| Zadania asynchroniczne | Celery | 5.x |
| Baza danych | PostgreSQL (Row-Level Security) | 16 |
| Broker wiadomości | RabbitMQ | 3.12 |
| Cache / Result Backend | Redis | 7 |
| Magazyn obiektowy | MinIO (S3-compatible) | — |
| Monitoring | Prometheus + Grafana + Loki + Alertmanager | — |
| Parsowanie kodu | tree-sitter (adaptery per język) | — |
| Szablony testów | Jinja2 | — |
| Uwierzytelnianie | Argon2id (hasła) + RS256 JWT (tokeny) | — |
| Płatności | tpay (BLIK, karta, przelew) | — |

## 1.4. Model C4

Model C4 definiuje cztery poziomy abstrakcji:

| Poziom | Nazwa | Co pokazuje |
|---|---|---|
| 1 | Context | System w kontekście użytkowników i systemów zewnętrznych |
| 2 | Container | Kontenery wdrożeniowe (aplikacje, bazy, kolejki) |
| 3 | Component | Moduły wewnętrzne per kontener |
| 4 | Code | Klasy, interfejsy, wzorce projektowe |

## 1.5. Pliki źródłowe diagramów

Wszystkie diagramy przechowywane są w formacie PlantUML (`.puml`) w katalogu `diagrams/`. Renderowanie do PNG/SVG wymaga narzędzia `plantuml` lub wtyczki IDE.

```
docs/final/architecture/diagrams/
├── c4-context.puml              # C4 Level 1
├── c4-container.puml            # C4 Level 2
├── c4-component-m1.puml         # C4 Level 3 – M1 Analiza kodu
├── c4-component-m2.puml         # C4 Level 3 – M2 Analiza dokumentacji
├── c4-component-m3.puml         # C4 Level 3 – M3 Analiza UML
├── c4-component-m4.puml         # C4 Level 3 – M4 Analiza GUI
├── c4-component-m5.puml         # C4 Level 3 – M5 Generowanie testów
├── c4-component-m6.puml         # C4 Level 3 – M6 Bezpieczeństwo
├── c4-code-engine.puml          # C4 Level 4 – Dual-path engine
├── c4-code-pipeline.puml        # C4 Level 4 – Pipeline przetwarzania
├── c4-code-parsers.puml         # C4 Level 4 – Parsery języków
├── flow-end-to-end.puml         # Przepływ end-to-end
├── flow-analiza-kodu.puml       # Przepływ – analiza kodu
├── flow-bezpieczenstwo.puml     # Przepływ – bezpieczeństwo
├── deploy-single-node.puml      # Wdrożenie – Docker Compose
├── deploy-multi-node.puml       # Wdrożenie – Kubernetes
├── seq-rejestracja.puml         # Sekwencja – rejestracja
├── seq-analiza-kodu.puml        # Sekwencja – analiza kodu
├── seq-generowanie-testow.puml  # Sekwencja – generowanie testów
├── seq-platnosc.puml            # Sekwencja – płatność
└── seq-webhook.puml             # Sekwencja – webhook
```

---

## 2. C4 Level 1 — Diagram kontekstowy (Context)

> **Źródło PlantUML:** `diagrams/c4-context.puml`

### 2.1. Opis

Diagram kontekstowy przedstawia TestGEN jako centralny system w otoczeniu użytkowników (aktorów) i systemów zewnętrznych.

### 2.2. Aktorzy

| Aktor | Opis | Interakcja |
|---|---|---|
| **Analityk QA (Anna)** | Główny użytkownik platformy | Tworzy projekty, przesyła pliki, uruchamia analizy, generuje testy, eksportuje raporty |
| **Administrator (Marek)** | Administrator platformy / Owner firmy | Zarządza firmami, użytkownikami, konfiguracją, monitoruje system |

### 2.3. Systemy zewnętrzne

| System | Protokół | Kierunek | Cel integracji |
|---|---|---|---|
| **GitHub / GitLab** | HTTPS, Git | Dwukierunkowy | Klonowanie repozytoriów, odbiór webhooków (push, merge) |

| System | Protokół | Kierunek | Cel integracji |
|---|---|---|---|
| **tpay** | HTTPS REST API | Dwukierunkowy | Obsługa płatności: checkout, webhooki, tokenizacja kart/BLIK |
| **TestRail** | HTTPS REST API | Wychodzący | Eksport przypadków testowych do zewnętrznego TMS |
| **Jira / Azure DevOps** | HTTPS REST API | Dwukierunkowy | Synchronizacja wyników analiz, tworzenie zadań, integracja per projekt |
| **Serwer SMTP** | SMTP/TLS | Wychodzący | Wysyłka e-maili: weryfikacja konta, zaproszenia, alerty, faktury |
| **API LLM (Anthropic / OpenAI)** | HTTPS REST API | Wychodzący | Modele językowe: analiza semantyczna kodu/dokumentacji, generowanie testów |
| **NVD / GitHub Advisory / OSV** | HTTPS REST API | Wychodzący | Bazy podatności CVE: aktualizacja reguł skanowania bezpieczeństwa |

### 2.4. Kluczowe wnioski

- TestGEN jest systemem **samodzielnym** — nie zależy od dostępności systemów zewnętrznych (poza LLM w trybie `llm` / `hybrid`)
- Tryb `native` działa w pełni offline (bez wywołań API LLM)
- Integracje (Jira, TestRail, GitHub) są **opcjonalne** — konfigurowane per projekt
- Płatności obsługiwane wyłącznie przez tpay (polska bramka)

---

## 3. C4 Level 2 — Diagram kontenerów (Container)

> **Źródło PlantUML:** `diagrams/c4-container.puml`

### 3.1. Opis

Diagram kontenerów przedstawia wszystkie kontenery wdrożeniowe platformy TestGEN i ich wzajemne relacje.

### 3.2. Kontenery

| Kontener | Technologia | Rola | Port |
|---|---|---|---|
| **Frontend (SPA)** | Angular 21, nginx | Interfejs użytkownika — standalone components, Material 3, routing, formularze | 80 (nginx) |
| **Shell Flutter** | Flutter WebView | Opakowanie SPA w aplikację natywną (mobile: Android/iOS, desktop: macOS/Windows/Linux) | — |
| **Backend API** | Python 3.12, FastAPI | REST API `/api/v1` — autoryzacja JWT, logika domenowa, walidacja, RLS per tenant | 8000 |

| Kontener | Technologia | Rola | Port |
|---|---|---|---|
| **Celery Workers** | Python 3.12, Celery 5 | Zadania asynchroniczne: analiza kodu, dokumentacji, UML, GUI; generowanie testów; skanowanie bezpieczeństwa; eksport raportów | — |
| **Celery Beat** | Python 3.12, Celery 5 | Harmonogram zadań periodycznych: odnowienia subskrypcji (03:30), snapshoty użycia (00:00), czyszczenie plików tymczasowych | — |
| **PostgreSQL** | PostgreSQL 16 | Relacyjna baza danych z Row-Level Security — dane biznesowe, wyniki analiz (JSONB), audyt | 5432 |
| **RabbitMQ** | RabbitMQ 3.12 | Broker wiadomości AMQP — kolejki: `default`, `analysis`, `export`, `security` | 5672 |
| **Redis** | Redis 7 | Cache sesji, result backend Celery, rate limiting, tymczasowe dane | 6379 |
| **MinIO** | MinIO | Magazyn obiektowy S3 — pliki źródłowe, dokumenty, raporty PDF, faktury VAT, zrzuty GUI | 9000 |

## 3.3. Protokoły komunikacji

| Źródło | Cel | Protokół | Opis |
|---|---|---|---|
| Frontend → Backend | REST | HTTP/HTTPS | JWT w HttpOnly cookie, format JSON |
| Backend → PostgreSQL | SQL | asyncpg (binary) | Asynchroniczne połączenia, RLS context per żądanie |
| Backend → RabbitMQ | AMQP | AMQP 0-9-1 | Kolejkowanie zadań Celery |
| Backend → Redis | Redis protocol | TCP 6379 | Cache, sesje, rate limiting |
| Backend → MinIO | S3 API | HTTP | Przechowywanie i pobieranie plików |
| RabbitMQ → Workers | AMQP | AMQP 0-9-1 | Dostarczanie zadań asynchronicznych |
| Workers → PostgreSQL | SQL | asyncpg | Zapis wyników analiz |
| Workers → Redis | Redis protocol | TCP 6379 | Zapis wyników zadań (result backend) |
| Workers → MinIO | S3 API | HTTP | Odczyt plików źródłowych, zapis raportów |
| Workers → API LLM | REST | HTTPS | Wywołania modeli językowych |

## 3.4. Kolejki Celery

| Kolejka | Przeznaczenie | Priorytet |
|---|---|---|
| `default` | Zadania ogólne (e-mail, czyszczenie, notyfikacje) | Normalny |

| Kolejka | Przeznaczenie | Priorytet |
|---|---|---|
| `analysis` | Analiza kodu, dokumentacji, UML, GUI | Wysoki |
| `export` | Generowanie raportów PDF/HTML/Excel | Normalny |
| `security` | Skanowanie bezpieczeństwa, aktualizacja CVE | Normalny |

---

# 4. C4 Level 3 — Diagramy komponentów (Component)

## 4.1. Moduł M1 — Analiza kodu

> **Źródło PlantUML:** `diagrams/c4-component-m1.puml`

**Odpowiedzialność:** Parsowanie i analiza kodu źródłowego — ekstrakcja struktury, zależności, logiki biznesowej.

**Komponenty:**

| Komponent | Technologia | Rola |
|---|---|---|
| Upload Handler | FastAPI route | Odbiera pliki kodu, waliduje format i rozmiar, zapisuje w MinIO |
| AST Parser | tree-sitter | Parsuje kod do drzewa składniowego (Abstract Syntax Tree) |
| Adaptery językowe | tree-sitter-{lang} | Ekstrakcja klas, funkcji, importów per język (Python, Java, JS/TS, C#, C/C++, PHP) |
| Native Analysis | Python | Analiza deterministyczna: złożoność cyklomatyczna, zależności modułów, struktury logiczne, przepływ danych |
| LLM Analysis | Python + API LLM | Analiza semantyczna via model językowy: logika biznesowa, przepływy, przypadki krawędziowe |
| LLM Gateway | Python | Abstrakcja dostawcy LLM (Anthropic / OpenAI) — routing, retry, rate limiting |
| Result Merger | Python | Scalanie wyników native + LLM: deduplikacja, priorytetyzacja, rozwiązywanie konfliktów |
| Analysis Storage | Python + SQLAlchemy | Zapis wyników do PostgreSQL (JSONB) i MinIO (pliki tymczasowe) |

**Obsługiwane języki programowania:**

| Język | Parser | Adapter |
|---|---|---|
| Python | tree-sitter-python | `ast/python_adapter.py` |
| Java | tree-sitter-java | `ast/java_adapter.py` |
| JavaScript | tree-sitter-javascript | `ast/javascript_adapter.py` |
| TypeScript | tree-sitter-typescript | `ast/typescript_adapter.py` |

| Język | Parser | Adapter |
|---|---|---|
| C# | tree-sitter-c-sharp | `ast/csharp_adapter.py` |
| C / C++ | tree-sitter-c / tree-sitter-cpp | `ast/c_adapter.py`, `ast/cpp_adapter.py` |
| PHP | tree-sitter-php | `ast/php_adapter.py` |

**Przepływ:**

1. Użytkownik przesyła pliki → Upload Handler → MinIO + DB
2. Parser AST wykrywa język i parsuje kod
3. Adapter językowy ekstrahuje elementy (klasy, funkcje, importy)
4. Zależnie od `engine_mode`:
   - `native` → analiza deterministyczna
   - `llm` → wywołanie API LLM
   - `hybrid` → oba + scalenie
5. Wyniki zapisywane w PostgreSQL (JSONB `analysis_results.result_data`)

---

## 4.2. Moduł M2 — Analiza dokumentacji

> **Źródło PlantUML:** `diagrams/c4-component-m2.puml`

**Odpowiedzialność:** Parsowanie dokumentów i ekstrakcja wymagań — funkcjonalnych i niefunkcjonalnych.

**Komponenty:**

| Komponent | Technologia | Rola |
|---|---|---|
| Document Upload | FastAPI route | Odbiera dokumenty w formatach PDF, DOCX, HTML, MD, TXT |
| Document Classifier | Python | Klasyfikacja typu dokumentu: specyfikacja, wymagania, podręcznik, notatki |
| PDF Parser | pdfplumber | Ekstrakcja tekstu, tabel i metadanych z plików PDF |
| DOCX Parser | python-docx | Ekstrakcja tekstu, struktury nagłówków i tabel z Word |
| HTML/MD Parser | BeautifulSoup / markdown | Ekstrakcja tekstu ze stron HTML i Markdown |
| Requirement Extractor | Python NLP | Ekstrakcja wymagań: identyfikacja zdań wymaganiowych, priorytet, kategoria |
| Glossary Extractor | Python NLP | Ekstrakcja terminów i definicji — budowanie glosariusza projektu |
| Inconsistency Detector | Python | Wykrywanie niespójności między dokumentami — sprzeczne wymagania, brakujące definicje |

| Komponent | Technologia | Rola |
|---|---|---|
| LLM Document Analyzer | Python + API LLM | Semantyczna analiza dokumentów: rozumienie kontekstu, intencji, relacji |

**Obsługiwane formaty:** PDF, DOCX, HTML, Markdown, TXT

---

### 4.3. Moduł M3 — Analiza UML

> **Źródło PlantUML:** `diagrams/c4-component-m3.puml`

**Odpowiedzialność:** Parsowanie diagramów UML i ekstrakcja elementów modelowych.

**Komponenty:**

| Komponent | Technologia | Rola |
|---|---|---|
| XMI Parser | Python lxml | Parsowanie diagramów w formacie XMI (eksport z narzędzi UML) |
| PlantUML Parser | Python regex | Parsowanie diagramów w formacie PlantUML |
| Visual Parser | Python + OCR/CV | Rozpoznawanie diagramów z obrazów (SVG, PNG) — OCR + computer vision |
| Class Diagram Analyzer | Python | Analiza klas: atrybuty, metody, relacje, dziedziczenie |
| Sequence Diagram Analyzer | Python | Analiza sekwencji: przepływy, komunikaty, aktorzy |
| Use-Case Analyzer | Python | Analiza przypadków użycia: aktorzy, scenariusze, inkluzje |
| State Machine Analyzer | Python | Analiza maszyn stanów: przejścia, warunki, akcje |
| Cross-Diagram Analyzer | Python | Korelacja między diagramami: spójność, pokrycie, brakujące elementy |
| LLM UML Analyzer | Python + API LLM | Semantyczna rekonstrukcja przepływów z diagramów |

**Obsługiwane formaty:** XMI, PlantUML, SVG, PNG (z OCR)

---

### 4.4. Moduł M4 — Analiza GUI

> **Źródło PlantUML:** `diagrams/c4-component-m4.puml`

**Odpowiedzialność:** Rozpoznawanie elementów interfejsu graficznego i rekonstrukcja przepływów użytkownika.

**Komponenty:**

| Komponent | Technologia | Rola |
|---|---|---|
| OCR Engine | Tesseract | Rozpoznawanie tekstu z elementów GUI |
| DOM Extractor | Selenium | Ekstrakcja struktury DOM ze stron webowych (live) |
| Component Detector | Computer Vision | Identyfikacja elementów UI: przyciski, formularze, tabele, menu |
| Flow Reconstructor | Python | Rekonstrukcja przepływów użytkownika z sekwencji ekranów |
| Visual Regression | pixelmatch | Porównanie wizualne: wykrywanie zmian UI między wersjami |
| LLM GUI Analyzer | Python + API LLM | Semantyczna identyfikacja elementów i interakcji |

---

## 4.5. Moduł M5 — Generowanie testów

> **Źródło PlantUML:** `diagrams/c4-component-m5.puml`

**Odpowiedzialność:** Generowanie przypadków testowych i kodu testowego na podstawie wyników analiz.

**Komponenty:**

| Komponent | Rola |
|---|---|
| Test Case Generator | Orkiestracja generowania — wybór strategii (native/LLM/hybrid) |
| Native Generator | Szablonowe generowanie przypadków na podstawie reguł (Jinja2) |
| LLM Generator | Semantyczne generowanie przypadków via model AI |
| Prioritizer | Priorytetyzacja testów: krytyczny / wysoki / średni / niski |
| Gherkin / BDD | Generowanie scenariuszy Given-When-Then |
| Code Generator | Renderowanie kodu testowego per framework (wzorzec strategii) |

**Obsługiwane frameworki testowe:**

| Kategoria | Framework | Język |
|---|---|---|
| Jednostkowe | pytest | Python |
| Jednostkowe | JUnit 5 | Java |
| Jednostkowe | NUnit | C# |
| Jednostkowe | Mocha / Jest | JavaScript/TypeScript |
| Jednostkowe | PHPUnit | PHP |
| E2E Web | Playwright | Python / JS / TS |
| E2E Web | Cypress | JavaScript |

| Kategoria | Framework | Język |
|---|---|---|
| E2E Web | Selenium | Python / Java / C# |
| E2E Mobile | Appium | Python / Java |
| Desktop GUI | SikuliX | Java (image-based) |
| Bazy danych | Database tests | SQL |

---

### 4.6. Moduł M6 — Analiza bezpieczeństwa

> **Źródło PlantUML:** `diagrams/c4-component-m6.puml`

**Odpowiedzialność:** Skanowanie bezpieczeństwa kodu — SAST, taint analysis, skanowanie zależności, raportowanie podatności.

**Komponenty:**

| Komponent | Rola |
|---|---|
| Dual-Path Orchestrator | Orkiestracja skanowania: reguły deterministyczne + LLM |
| SAST Scanner | Statyczna analiza kodu (reguły Semgrep/Bandit) |
| Taint Analysis | Śledzenie przepływu danych: SQL injection, XSS, path traversal |
| Dependency Scanner | Skanowanie zależności: bazy CVE, ocena ryzyka |
| Config Scanner | Analiza konfiguracji: sekrety w kodzie, hardening |
| CVE Pipeline | Automatyczna aktualizacja bazy CVE (NVD, GitHub Advisory, OSV) |
| CVSS Calculator | Obliczanie wyników CVSS per podatność |
| LLM SAST | Semantyczna analiza bezpieczeństwa via AI |
| Security Report | Raport podatności: klasyfikacja Critical/High/Medium/Low |
| SBOM Generator | Software Bill of Materials (CycloneDX / SPDX) |

**Bramka funkcji:** moduł bezpieczeństwa dostępny wyłącznie w planie Premium.

---

### 4.7. Moduł płatności

**Odpowiedzialność:** Obsługa subskrypcji, płatności, faktur VAT i limitów.

**Komponenty:**

| Komponent | Rola |
|---|---|
| Checkout Service | Tworzenie transakcji tpay, generowanie URL płatności |
| Webhook Processor | Odbiór i przetwarzanie webhooków tpay (JWS) |
| Subscription Service | Zarządzanie subskrypcjami: tworzenie, odnowienie, dunning |
| Invoice Service | Generowanie faktur VAT (PDF) → MinIO |

| Komponent | Rola |
|---|---|
| Quota Service | Egzekwowanie limitów: analizy/mies., test case'y/mies., storage |
| Feature Gating Service | Bramki funkcji: weryfikacja dostępu per plan |
| Payment Token Store | Przechowywanie tokenów płatniczych (BLIK alias, karta) |

---

### 4.8. Moduł integracji

**Odpowiedzialność:** Integracja z systemami zewnętrznymi (per projekt).

**Komponenty:**

| Komponent | System | Rola |
|---|---|---|
| Jira Adapter | Jira Cloud | Tworzenie zadań, synchronizacja statusów |
| Azure DevOps Adapter | Azure DevOps | Tworzenie work items, integracja z pipeline'ami |
| TestRail Exporter | TestRail | Eksport przypadków testowych |
| Jira XRay Exporter | Jira XRay | Eksport testów w formacie XRay |

---

## 5. C4 Level 4 — Diagramy kodu (Code)

### 5.1. Dual-Path Engine — wzorzec strategii

> **Źródło PlantUML:** `diagrams/c4-code-engine.puml`

**Opis:** Centralny wzorzec architektoniczny TestGEN — silnik analizy obsługujący trzy tryby przetwarzania.

**Klasy:**

| Klasa | Odpowiedzialność |
|---|---|
| `AnalysisStrategy` (interfejs) | Kontrakt: `analyze(source_files, config) → AnalysisResult` |
| `NativeStrategy` | Analiza deterministyczna — AST + reguły + heurystyki (bez API LLM, zero kosztów) |
| `LLMStrategy` | Analiza semantyczna — wywołanie modelu językowego (Anthropic / OpenAI) |
| `HybridStrategy` | Oba podejścia + scalenie wyników (ResultMerger) |
| `AnalysisEngine` | Orkiestrator — wybiera strategię na podstawie `engine_mode` z konfiguracji |
| `ResultMerger` | Scalanie wyników: deduplikacja, priorytetyzacja, rozwiązywanie konfliktów |

**Enum `EngineMode`:**

- `NATIVE` — tylko analiza deterministyczna
- `LLM` — tylko model językowy
- `HYBRID` — oba + scalenie

**Wzorzec ten stosowany jest w:**

- Analiza kodu (M1) — `services/analysis/native_analysis.py` + `llm_analysis.py`
- Analiza dokumentacji (M2) — `services/analysis/document/requirement_extractor.py` + `llm_analysis.py`
- Generowanie testów (M5) — `services/test_generation/` (native Jinja2 + LLM)
- Skanowanie bezpieczeństwa (M6) — `services/security_scan/dual_path_orchestrator.py`
- Gap Analysis — `services/gap_analysis/dual_path_orchestrator.py`

---

## 5.2. Pipeline przetwarzania — wzorzec potoku

> **Źródło PlantUML:** `diagrams/c4-code-pipeline.puml`

**Opis:** Przetwarzanie analiz odbywa się sekwencyjnie przez kroki potoku (pipeline), z kontekstem przekazywanym między krokami.

**Klasy:**

| Klasa | Odpowiedzialność |
|---|---|
| `PipelineStep` (interfejs) | Kontrakt: `execute(context) → PipelineContext` |
| `PipelineContext` | Obiekt przenoszący stan: pliki, wyniki parsowania, wyniki analiz, test case'y, błędy |
| `ParseStep` | Parsowanie plików (AST, dokumenty, UML, GUI) |
| `AnalyzeStep` | Uruchomienie dual-path analizy |
| `GenerateStep` | Generowanie przypadków testowych i kodu |
| `SecurityStep` | Skanowanie bezpieczeństwa |
| `StoreStep` | Zapis wyników do bazy i magazynu plików |
| `AnalysisPipeline` | Orkiestrator — sekwencyjne wykonanie kroków z obsługą błędów |

**Przepływ:**

```
ParseStep → AnalyzeStep → GenerateStep → SecurityStep → StoreStep
```

Każdy krok otrzymuje `PipelineContext` i zwraca wzbogacony kontekst. Błąd w dowolnym kroku jest rejestrowany w `context.errors`, a przetwarzanie może kontynuować (graceful degradation) lub zatrzymać się (fail-fast), zależnie od konfiguracji.

---

### 5.3. Parsery języków — wzorzec strategii

> **Źródło PlantUML:** `diagrams/c4-code-parsers.puml`

**Opis:** Każdy obsługiwany język programowania ma dedykowany parser oparty na tree-sitter.

**Klasy:**

| Klasa | Odpowiedzialność |
|---|---|
| `BaseParser` (abstrakcyjna) | Wspólna logika parsowania: inicjalizacja tree-sitter, ekstrakcja klas/funkcji/importów |
| `PythonParser` | Dodatkowa ekstrakcja: dekoratory, docstringi, type hints |
| `JavaParser` | Dodatkowa ekstrakcja: adnotacje, pakiety, interfejsy |
| `TypeScriptParser` | Dodatkowa ekstrakcja: interfejsy, typy, generyki |
| `CSharpParser` | Dodatkowa ekstrakcja: namespace'y, atrybuty, LINQ |
| `CppParser` | Dodatkowa ekstrakcja: nagłówki, szablony, makra |
| `PHPParser` | Dodatkowa ekstrakcja: traity, namespace'y |

**Zasada:** Nowy język = nowa klasa dziedzicząca `BaseParser` + odpowiednia gramatyka tree-sitter.

---

## 6. Diagramy przepływu danych

### 6.1. Przepływ end-to-end

> **Źródło PlantUML:** `diagrams/flow-end-to-end.puml`

Kompletny przepływ od przesłania plików do eksportu raportów:

```
Użytkownik
  │
  ├── 1. Tworzy projekt
  ├── 2. Przesyła pliki (kod, dokumentacja, UML, GUI)
  │
  ▼
Backend API
  │
  ├── 3. Walidacja i zapis plików (MinIO)
  ├── 4. Rejestracja metadanych (PostgreSQL)
  ├── 5. Kolejkowanie zadania (RabbitMQ)
  │
  ▼
Celery Worker
  │
  ├── 6. Analiza (M1-M4): kod, dokumentacja, UML, GUI
  ├── 7. Scalenie wyników
  ├── 8. Generowanie testów (M5): przypadki + kod
  ├── 9. Skanowanie bezpieczeństwa (M6)
  ├── 10. Gap Analysis: wymagania vs implementacja vs testy
  ├── 11. Zapis wyników (PostgreSQL)
  │
  ▼
Użytkownik
  │
  ├── 12. Przegląd wyników
  ├── 13. Edycja i zatwierdzenie testów
  └── 14. Eksport raportów (PDF/HTML/Excel/CSV/JSON)
```

### 6.2. Przepływ analizy kodu

> **Źródło PlantUML:** `diagrams/flow-analiza-kodu.puml`

**Etapy:**

1. Przesłanie plików → walidacja → MinIO + PostgreSQL
2. Kolejkowanie zadania → RabbitMQ (kolejka `analysis`)
3. Celery Worker pobiera pliki z MinIO
4. Wykrywanie języka i frameworka
5. Rozgałęzienie wg `engine_mode`:
   - **native:** parsowanie AST (tree-sitter) → analiza deterministyczna (złożoność, zależności, logika, przepływ danych)
   - **llm:** przygotowanie promptu → wywołanie API LLM → parsowanie odpowiedzi
   - **hybrid:** native + LLM + scalenie (ResultMerger)
6. Zapis wyników → PostgreSQL (JSONB) + aktualizacja statusu
7. Powiadomienie użytkownika

### 6.3. Przepływ skanowania bezpieczeństwa

> **Źródło PlantUML:** `diagrams/flow-bezpieczenstwo.puml`

**Etapy:**

1. Sprawdzenie bramki funkcji (plan Premium wymagany)
2. Kolejkowanie → RabbitMQ (kolejka `security`)
3. Równoległe skanery: SAST, Taint Analysis, Dependency Scan, Config Scan
4. Obliczenie wyników CVSS per podatność
5. Klasyfikacja: Critical / High / Medium / Low
6. Opcjonalnie: LLM SAST (semantyczna weryfikacja)
7. Generowanie raportu + SBOM (CycloneDX/SPDX)
8. Zapis wyników → PostgreSQL

---

## 7. Diagramy wdrożeniowe (Deployment)

### 7.1. Single-node — Docker Compose

> **Źródło PlantUML:** `diagrams/deploy-single-node.puml`

**Scenariusz:** Jeden serwer, wszystkie kontenery na jednej maszynie.

**Wymagania minimalne:**

- 4 vCPU, 16 GB RAM, 100 GB SSD
- Ubuntu 22.04+ / Debian 12+
- Docker 24+ z Compose v2

**Kontenery:**

| Kontener | Obraz | Porty | Wolumeny |
|---|---|---|---|
| nginx | nginx:alpine | 80, 443 | certyfikaty SSL |
| frontend | testgen-frontend | — (proxy przez nginx) | — |
| backend | testgen-backend | 8000 | — |
| celery-worker | testgen-backend | — | — |
| celery-beat | testgen-backend | — | — |
| flower | mher/flower | 5555 | — |
| postgres | postgres:16-alpine | 5432 | `pg-data` |
| redis | redis:7-alpine | 6379 | `redis-data` |
| rabbitmq | rabbitmq:3.12-management | 5672, 15672 | `rabbitmq-data` |
| minio | minio/minio | 9000, 9001 | `minio-data` |

**Sieć:** `testgen-net` (bridge) — izolacja kontenerów.

**Szczegóły konfiguracji:** Dokumentacja wdrożeniowa — instalacja

---

## 7.2. Multi-node — Kubernetes / Helm

> **Źródło PlantUML:** `diagrams/deploy-multi-node.puml`

**Scenariusz:** Klaster Kubernetes z wieloma node'ami, skalowanie horyzontalne.

**Przestrzenie nazw (Namespaces):**

| Namespace | Komponenty |
|---|---|
| `testgen` | frontend (Deployment), backend (Deployment), celery-worker (Deployment), celery-beat (Deployment) |
| `testgen-data` | PostgreSQL (StatefulSet + PVC), PgBouncer (Deployment), Redis + Sentinel (StatefulSet), RabbitMQ Cluster (StatefulSet), MinIO Distributed (StatefulSet + PVC) |
| `monitoring` | Prometheus, Grafana, Loki, Alertmanager |

**Skalowanie (HPA):**

| Komponent | Min replik | Max replik | Metryka wyzwalająca |
|---|---|---|---|
| Frontend | 2 | 4 | CPU > 70% |
| Backend API | 2 | 6 | CPU > 70% |
| Celery Workers | 2 | 8 | Głębokość kolejki > 100 |
| Celery Beat | 1 | 1 | Singleton |

**Baza danych:**

- PostgreSQL Primary (zapis + odczyt) + Replica (tylko odczyt, streaming replication)
- PgBouncer: connection pooling, `pool_mode=transaction`, `max_client_conn=400`

**Wysoka dostępność:**

- Redis Sentinel: automatyczny failover
- RabbitMQ: klaster 3 node'ów, quorum queues
- MinIO: tryb rozproszony (4+ node'y)

**Szczegóły konfiguracji:** Dokumentacja wdrożeniowa — instalacja (Kubernetes) i operacje (skalowanie)

---

## 8. Diagramy sekwencji

### 8.1. Rejestracja użytkownika

> **Źródło PlantUML:** `diagrams/seq-rejestracja.puml`

**Aktorzy:** Użytkownik, Frontend (Angular), Backend API (FastAPI), PostgreSQL, SMTP

**Przepływ:**

1. Użytkownik wypełnia formularz rejestracji (e-mail, hasło, nazwa firmy)
2. Frontend wysyła `POST /api/v1/auth/register`
3. Backend waliduje dane, sprawdza unikalność e-maila
4. W jednej transakcji tworzone są: tenant (firma), user (konto), user_tenant (powiązanie z rolą Owner), subscription (plan Freemium)
5. Hasło haszowane algorytmem Argon2id
6. Generowany jest token weryfikacyjny (JWT, 24h) → e-mail
7. Użytkownik klika link → `POST /api/v1/auth/verify-email` → konto aktywowane

**Kluczowe aspekty:**

- Atomowość: wszystkie wpisy tworzony w jednej transakcji
- Bezpieczeństwo: hasło nigdy nie trafia do bazy w postaci jawnej
- Audyt: rejestracja i weryfikacja logowane w `audit_logs`

---

### 8.2. Analiza kodu

> **Źródło PlantUML:** `diagrams/seq-analiza-kodu.puml`

**Aktorzy:** Użytkownik, Frontend, Backend API, RabbitMQ, Celery Worker, PostgreSQL, MinIO, API LLM

**Przepływ:**

1. Użytkownik przesyła pliki kodu → `POST /api/v1/projects/{id}/uploads`
2. Backend: walidacja → sprawdzenie kwot → zapis w MinIO → rejestracja w DB → kolejkowanie zadania
3. Odpowiedź 202 Accepted z `task_id`
4. Celery Worker: pobranie plików → wykrywanie języka → analiza (native/LLM/hybrid) → zapis wyników
5. Frontend odpytuje status (`GET /api/v1/tasks/{task_id}`) co 3 sekundy
6. Po zakończeniu: wyświetlenie wyników

**Kluczowe aspekty:**

- Asynchroniczność: analiza odbywa się w Celery Worker, nie blokuje API
- Quota enforcement: sprawdzenie limitów przed kolejkowaniem
- Polling: frontend odpytuje status zadania (alternatywa: WebSocket)

---

### 8.3. Generowanie testów

> **Źródło PlantUML:** `diagrams/seq-generowanie-testow.puml`

**Przepływ:**

1. Użytkownik wybiera ukończoną analizę → `POST /api/v1/projects/{id}/test-cases/generate`
2. Backend: walidacja → bramka funkcji → kolejkowanie
3. Celery Worker: pobranie wyników analizy → generowanie (native/LLM/hybrid) → priorytetyzacja → zapis test case'ów → renderowanie kodu per framework
4. Wyniki: test case'y (status `draft`) + kod testowy per framework

**Workflow statusów test case'ów:**

```
draft → review → approved → exported
```

---

### 8.4. Płatność (checkout + webhook)

> **Źródło PlantUML:** `diagrams/seq-platnosc.puml`

**Przepływ — checkout:**

1. Owner wybiera plan → `POST /api/v1/billing/checkout`
2. Backend tworzy transakcję w tpay → `POST /transactions`
3. Użytkownik przekierowywany na stronę płatności tpay (BLIK/karta/przelew)

**Przepływ — webhook:**

1. tpay wysyła webhook (`POST /api/v1/billing/webhooks/tpay`) z podpisem JWS
2. Backend weryfikuje podpis → zapisuje zdarzenie → klasyfikuje
3. Transakcja `paid` → aktualizacja planu + generowanie faktury VAT + zapis w MinIO
4. Tokenizacja → zapis tokenu płatniczego do automatycznych odnowień

**Przepływ — odnowienie (Celery Beat, 03:30):**

1. Pobranie subskrypcji do odnowienia
2. Obciążenie zapisanego tokenu
3. Sukces → przedłużenie + faktura; Porażka → okres łaski (7 dni) → downgrade

---

### 8.5. Przetwarzanie webhooków (ogólny)

> **Źródło PlantUML:** `diagrams/seq-webhook.puml`

**Wzorzec uniwersalny** stosowany dla wszystkich webhooków (tpay, GitHub):

1. Odbiór podpisanego payloadu
2. Weryfikacja podpisu (JWS / HMAC-SHA256)
3. Zapis surowego zdarzenia w `webhook_events` (z `idempotency_key`)
4. Sprawdzenie idempotentności — duplikaty ignorowane (200 OK)
5. Klasyfikacja zdarzenia (`WebhookProcessor.classify()`)
6. Przetwarzanie: synchroniczne (proste aktualizacje) lub asynchroniczne (Celery)
7. Aktualizacja statusu zdarzenia → `processed`

**Gwarancje:**

- **Idempotentność:** wielokrotne dostarczenie tego samego webhooka nie powoduje podwójnego efektu
- **Audytowalność:** każde zdarzenie zapisane w bazie
- **Bezpieczeństwo:** weryfikacja podpisu odrzuca sfałszowane żądania

---

## 9. Decyzje architektoniczne (ADR)

Kluczowe decyzje architektoniczne udokumentowane w adr/:

| ADR | Decyzja | Uzasadnienie |
|---|---|---|
| ADR-001 | Monorepo (backend + frontend + flutter) | Współdzielenie konfiguracji, atomowe commity, jeden CI pipeline |
| ADR-002 | Multi-tenancy via PostgreSQL RLS | Izolacja danych na poziomie DB, brak logiki filtrowania w kodzie |
| ADR-003 | Argon2id + RS256 JWT | Najsilniejszy algorytm haszowania + asymetryczne JWT |
| ADR-004 | Celery + RabbitMQ | Niezawodne kolejkowanie, routing po typie zadania, retry z backoff |
| ADR-005 | tree-sitter jako parser AST | Incremental parsing, 40+ języków, C-level wydajność |
| ADR-006 | Jinja2 do szablonów testów | Elastyczność, dziedziczenie szablonów, makra |
| ADR-007 | Frameworki E2E (Playwright/Cypress/Selenium) | Pokrycie różnych potrzeb: modern, cross-browser, legacy |
| ADR-008 | Golden data (wzorcowe dane testowe) | Deterministyczne testy integracyjne, reproducibility |
| ADR-009 | Workflow code review | Zapewnienie jakości kodu, blokujące review |

| ADR | Decyzja | Uzasadnienie |
|---|---|---|
| ADR-010 | Audit logging | Śledzenie operacji wrażliwych, compliance |
| ADR-011 | UML Tier 2 (visual parsing) | Rozpoznawanie diagramów z obrazów (OCR + CV) |
| ADR-012 | SikuliX (testy GUI desktop) | Testy oparte na obrazach dla aplikacji desktopowych |
| ADR-013 | Traceability proxy | Śledzenie powiązań: wymagania ↔ implementacja ↔ testy |

---

# 10. Konwencje diagramów

## 10.1. Style PlantUML

Diagramy C4 korzystają z biblioteki `C4-PlantUML` (GitHub: plantuml-stdlib/C4-PlantUML):

- **Niebieskie:** komponenty wewnętrzne TestGEN
- **Szare:** systemy zewnętrzne
- **Postacie:** aktorzy (użytkownicy)
- **Cylindry:** bazy danych i magazyny
- **Prostokąty z zaokrąglonymi rogami:** kolejki wiadomości

## 10.2. Nazewnictwo plików

| Prefiks | Typ diagramu |
|---|---|
| `c4-context-` | C4 Level 1 |
| `c4-container-` | C4 Level 2 |
| `c4-component-` | C4 Level 3 |
| `c4-code-` | C4 Level 4 |
| `flow-` | Przepływ danych |
| `deploy-` | Diagram wdrożeniowy |
| `seq-` | Diagram sekwencji |

### 10.3. Renderowanie

```
# Renderowanie pojedynczego diagramu
plantuml -tpng diagrams/c4-context.puml

# Renderowanie wszystkich diagramów do SVG
plantuml -tsvg diagrams/*.puml

# Renderowanie z Custom Theme
plantuml -tpng -theme C4_blue diagrams/*.puml
```

Wyrenderowane pliki umieść w `diagrams/render/` (katalog nie jest wersjonowany w repozytorium — generowany ad hoc).

---

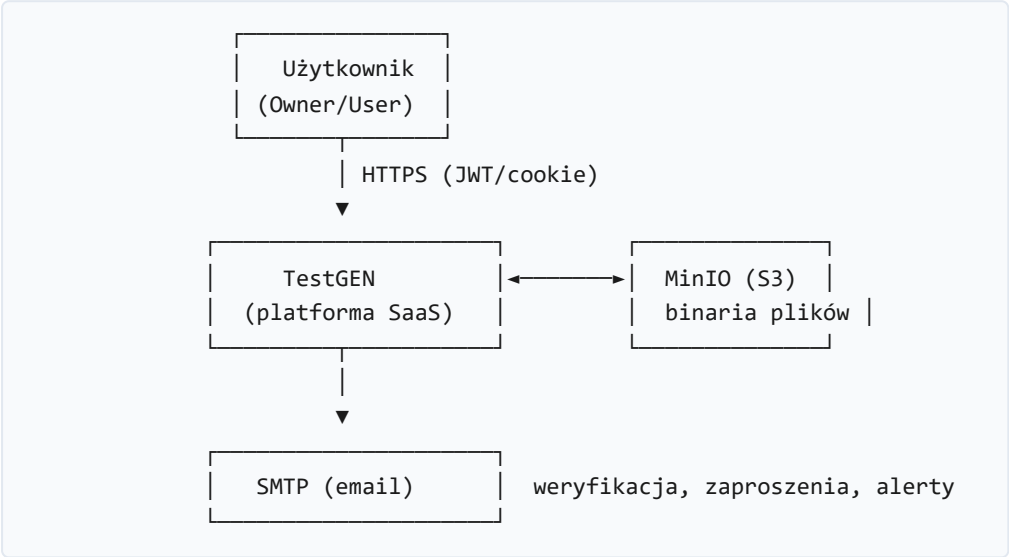
## Powiązane dokumenty

- system.md — uproszczony opis architektury (tekst + ASCII art)
- conventions.md — konwencje kodu i dokumentacji
- workflows.md — przepływy pracy deweloperskiej
- adr/ — decyzje architektoniczne (ADR)
- api/overview.md — przegląd API REST
- billing/ — architektura modułu płatności
- interfejs-webowy/ — architektura frontend
- Dokumentacja wdrożeniowa — instalacja i operacje
- Podręcznik administratora — operacje administracyjne

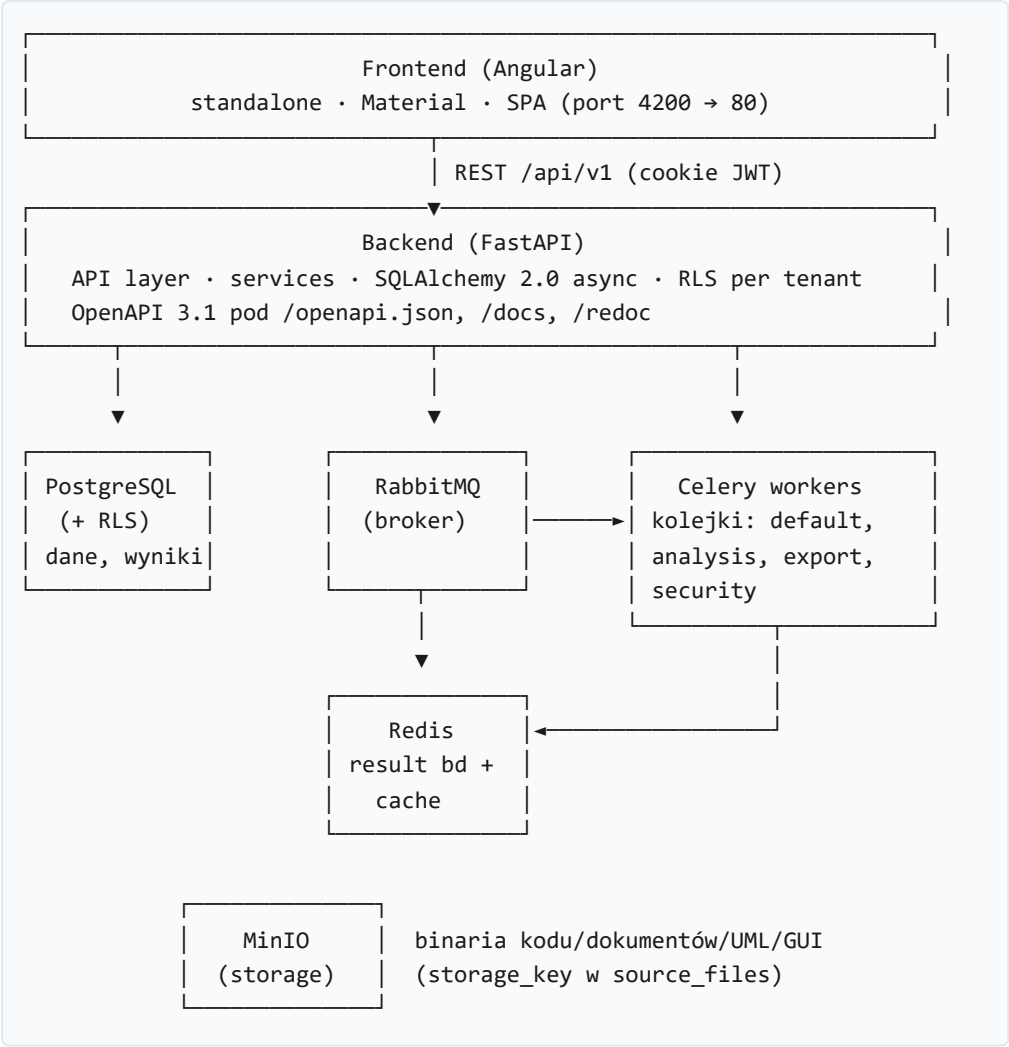
# Architektura — TestGEN (P1)

Architektura warstwowa platformy. Decyzje architektoniczne: adr/. Diagram C4 (uproszczony, tekstowy).

## C4 — Context



## C4 — Container




## Warstwy backendu

- **API** (`app/api/routes/`) — endpointy FastAPI, autoryzacja JWT/cookie, wymuszanie RLS, walidacja Pydantic, błędy RFC 7807.
- **Serwisy** (`app/services/`) — logika domenowa: analiza dokumentacji (`analysis/document/`), analiza kodu (AST), generowanie testów, auth, quota, audyt.
- **Workery** (`app/workers/`, Celery) — zadania asynchroniczne: parsowanie dokumentów, analiza, czyszczenie, export, skany. Patrz ADR-004.
- **Modele** (`app/models/`, SQLAlchemy 2.0 async) — PostgreSQL z RLS; `TenantMixin` (tenant_id), wyniki analiz w `AnalysisResult.result_data` (JSONB).

## Przepływ asynchroniczny (przykład: analiza dokumentacji)

```
POST /projects/{id}/upload-documents
  → MinIO (binaria) + SourceFile(processing)
  → parse_document_task.delay()  → 202 + task_id

Celery worker:
  → parser formatu → ParsedDocument
  → preprocessing → AnalysisResult(document_preprocessing)
  → classify / requirements / glossary / inconsistencies
  → gap_analysis / traceability / test_generation
  → SourceFile(ready)

GET /documents/{id}/status  → available_results rośnie z każdym modułem
```

Każdy moduł utrwala wynik niezależnie — częściowe wyniki dostępne przed zakończeniem pipeline'u. Błąd jednego modułu nie blokuje pozostałych (non-fatal).

## Izolacja danych (RLS)

Każda tabela dziedziczy `tenant_id`; przed zapytaniem sesja ustawia `app.current_tenant`. Polityki RLS wymuszają, że wiersz jest widoczny tylko dla swojego tenanta — nawet pominięcie filtru w ORM nie ujawnia danych (ADR-002).

## Powiązane

- getting-started.md — uruchomienie środowiska.
- environment-setup.md — serwisów i porty.
- ai/document-analysis.md, ai/ast-analysis.md — szczegóły modułów analizy.

# Konwencje kodu — TestGEN (P1)

## Nazewnictwo

| Element | Konwencja | Przykład |
|---|---|---|
| Plik Python | `snake_case.py` | `requirement_extractor.py` |
| Klasa Python | `PascalCase` | `RequirementExtractor` |
| Komponent Angular | `kebab-case.component.ts` | `document-view.component.ts` |
| Endpoint REST | `kebab-case`, rzeczownik w l. mnogiej | `/api/v1/test-cases` |
| Tabela SQL | `snake_case`, l. mnoga | `source_files`, `analysis_results` |
| Zmienna środowiskowa | `UPPER_SNAKE` | `DATABASE_URL` |
| Branch gita | `feature/*`, `fix/*`, `hotfix/*`, `docs/*` | `feature/2-13-docs-api` |

## Struktura katalogów

```
backend/
  app/
    api/routes/        # routery FastAPI (per domena)
    services/          # logika domenowa (analysis/, auth/, ...)
    models/            # modele SQLAlchemy (RLS, TenantMixin)
    schemas/           # schematy Pydantic
    workers/           # zadania Celery (tasks/, celery_app.py)
    core/              # config, database, security, cookies, storage
  alembic/versions/    # migracje
  tests/               # unit / integration / golden_data
frontend/
  src/app/             # standalone components, Material, signals
docs/final/            # ta dokumentacja (user-guide, deployment-guide,
                       # admin-guide, architecture, ai-modules,
                       # troubleshooting, release-notes)
```

## Testy

| Typ | Lokalizacja | Narzędzie |
|---|---|---|
| Jednostkowe (backend) | `backend/tests/unit/` | pytest |
| Integracyjne (backend) | `backend/tests/integration/` | pytest |
| Golden data | `backend/tests/golden_data/` | pytest, marker `@golden` |
| Frontend | `frontend/` | Vitest/Karma |

- Pokrycie backendu ≥ 80% (bramka CI: `--cov-fail-under=80`).
- Każda nowa funkcjonalność wymaga testów jednostkowych + integracyjnych.
- Zmiana zachowania analizy = aktualizacja ground truth, nie obniżenie progu (ADR-008).

## Git flow

- `main` — stabilna, produkcja.
- `develop` — integracja.
- `feature/*`, `fix/*`, `hotfix/*`, `docs/*` — praca tematyczna.
- PR wymaga **min. 1 approve** i zielonego CI.

## Commity

**Conventional Commits z emoji** (zgodnie z historią repo i `commitlint`):

```
✨ feat(scope): krótki opis po polsku (Story X.Y, ACn)
🦎 fix(scope): opis poprawki
📝 docs(p1): opis zmiany dokumentacji
♻️ refactor(scope): opis
✅ test(scope): opis
```

- Język komunikatów: polski dla opisu, angielski dla typu (`feat` / `fix` /...).
- Referencja do story i AC w treści (np. `Story 2.13, AC1`).
- **Bez `--no-verify`** — pre-commit `ruff` / `ruff-format` auto-naprawia pliki; przy niepowodzeniu: `git add -A` i ponowić commit.

## Linting / formatowanie

- Backend: `ruff check`, `ruff format` (`make lint`, `make format`).
- Frontend: `ng lint`, Prettier.
- Reguła TC001 egzekwowana — importy type-only pod `if TYPE_CHECKING:`.

## Code review

- Min. 1 approve, CI green.
- Nowy endpoint → polski `summary` / `description` + przykłady w OpenAPI (api/overview.md).
- Nowa tabela → `tenant_id` + polityka RLS + migracja Alembic.

# Workflow dla deweloperów — TestGEN (P1)

Przepisy na najczęstsze zadania rozwojowe. Konwencje: conventions.md.

## Dodanie nowego języka parsera (AST)

Analiza kodu oparta na tree-sitter (ADR-005).

1. Dodaj grammar tree-sitter dla języka w module parserów ( `backend/app/` obsługa języków).
2. Zarejestruj język w rejestrze parserów (mapowanie rozszerzenie → grammar).
3. Dodaj próbki golden data (kod + oczekiwane `logic_structures` ).
4. Testy: ekstrakcja funkcji/klas/importów/parametrów.
5. Dokumentacja: zaktualizuj ai/ast-analysis.md.

## Dodanie nowego frameworku testowego

Generowanie kodu przez Jinja2 (ADR-006).

1. Utwórz szablon Jinja2 w katalogu szablonów testowych (jednostkowy lub E2E — patrz ADR-007).
2. Zarejestruj framework w rejestrze frameworków.
3. Dodaj próbki golden data: scenariusz → oczekiwany kod.
4. Testy regresyjne stabilności szablonu.
5. Dokumentacja: zaktualizuj ai/test-code-generation.md.

## Dodanie nowego formatu dokumentacji

Pipeline natywny (ai/document-analysis.md).

1. Utwórz parser w `backend/app/services/analysis/document/` zwracający `ParsedDocument` .
2. Zarejestruj rozszerzenie w `get_document_parser()` .
3. Dodaj detekcję nagłówków w preprocesorze ( `DocumentPreprocessor` ).
4. Próbki golden data (dokument + ground truth sekcji/wymagań).
5. Testy + zaktualizuj ai/document-formats.md.

## Dodanie nowej reguły analizy dokumentacji

1. Zaimplementuj regułę w odpowiednim serwisie ( `requirement_extractor` , `inconsistency_detector` , `glossary_extractor` ...).
2. Dodaj próbkę golden data weryfikującą regułę.
3. Zaktualizuj progi w `docs_thresholds.json` jeśli reguła zmienia liczności.
4. Test + dokumentacja w ai/document-analysis.md.

## Dodanie nowej tabeli (z RLS)

1. Model SQLAlchemy z `TenantMixin` (`tenant_id`) w `backend/app/models/`.
2. Migracja Alembic + **polityka RLS** na tabeli.
3. Schemat Pydantic + router (polski `summary` / `description`, przykłady).
4. Testy z ustawionym kontekstem `app.current_tenant`.

## Uruchomienie zadania asynchronicznego lokalnie

```
make celery-worker          # worker kolejki default
make celery-beat            # scheduler (zadania cykliczne)
make celery-flower          # UI monitoringu (http://localhost:5555)
make celery-inspect         # aktywne zadania
```

Kolejki tematyczne: `default`, `analysis`, `export`, `security` (ADR-004).

## Aktualizacja golden data przy celowej zmianie

1. Zmień kod analizy.
2. Wygeneruj na nowo ground truth: `_generate_samples.py`, `_generate_ground_truth.py`.
3. Zweryfikuj metryki vs progi (`pytest tests/golden_data/ -m golden -v`).
4. Zaktualizuj ai/benchmarks.md po release.

# ADR-001: Custom monorepo (FastAPI + Angular + Flutter)

## Status

Accepted

## Kontekst

Platforma TestGEN składa się z trzech powiązanych warstw: backend (FastAPI), frontend web (Angular) oraz klient desktop/mobile (Flutter WebView). Wymagana jest spójna organizacja kodu, jeden potok CI/CD oraz współdzielenie kontraktu API między warstwami. Rozproszenie kodu do osobnych repozytoriów wydłużałoby synchronizację zmian i utrudniało atomiczne commity (zmiana endpointu + aktualizacja klienta).

## Decyzja

Prowadzimy **własny monorepo** z trzema katalogami głównymi:

- `backend/` — FastAPI + SQLAlchemy 2.0 (async) + Celery,
- `frontend/` — Angular (standalone, Material),
- `mobile/` — Flutter WebView (klient desktop/mobile P3).

Infrastrukturę uruchamia `docker-compose.yml` (8 serwisów: backend, frontend, celery-worker, flower, postgres, redis, rabbitmq, minio). Kontrakt API współdzielony przez auto-generowaną specyfikację OpenAPI 3.1 z FastAPI, dostępną pod `/openapi.json` oraz interaktywnie pod `/docs` (Swagger UI) i `/redoc`.

## Konsekwencje

**Pozytywne:**

- Jeden potok CI/CD (`make ci-lint`, `make ci-test`, `make ci-build`) dla całej platformy; atomiczne commity obejmujące backend + frontend.
- Spójne wersjonowanie i jedno repozytorium źródła prawdy.
- Łatwiejszy onboarding — jedna struktura katalogów do poznania.

**Negatywne:**

- Większe repozytorium; clone i historia są cięższe niż w polyrepo.
- Współdzielenie typów wymaga dyscypliny (regeneracja klienta API z OpenAPI, nie ręczna edycja).
- Silniejsze powiązanie zmian między zespołami backend/frontend.

## Powiązane

- ADR-004 — worker Celery w tym samym obrazie co backend.
- architecture.md — diagram C4 kontenerów.

# ADR-002: Multi-tenancy z Row-Level Security (RLS)

## Status

Accepted

## Kontekst

Platforma obsługuje wielu tenantów (organizacji) potencjalnie z wrażliwym kodem źródłowym i dokumentacją. Wymagana jest silna izolacja danych między tenantami na poziomie bazy danych — poleganie wyłącznie na filtrowaniu w warstwie aplikacji (klauzula `WHERE tenant_id = ...` w każdym zapytaniu) jest podatne na błąd: jedno pominięcie filtru ujawnia dane sąsiada.

## Decyzja

Wielodostępność oparta na **shared database + Row-Level Security (RLS) PostgreSQL**. Każda tabela dziedziczy `tenant_id` (mixinx `TenantMixin`). Przed wykonaniem zapytania sesja ustawia zmienną transakcji `app.current_tenant = <uuid>`, a polityki RLS wymuszają, że wiersz jest widoczny tylko gdy `tenant_id` odpowiada bieżącej sesji. Migracje definiują polityki RLS per tabela.

## Konsekwencje

**Pozytywne:**

- Izolacja wymuszana przez bazę — nawet pominięcie filtru w ORM nie ujawni danych innego tenanta.
- Oszczędność zasobów: jedna baza, jeden schemat, wspólne indeksy.
- Spójny model dla wszystkich modułów analizy (kod, dokumentacja, UML/GUI).

**Negatywne:**

- Wymaga dyscypliny: każda nowa tabela musi mieć `tenant_id` + politykę RLS.
- Większe indeksy (z uwzględnieniem `tenant_id`).
- Utrudnione współdzielenie danych między tenantami (z założenia rzadkie).
- Złożoność testowania: testy muszą ustawiać kontekst sesji RLS.

## Powiązane

- ADR-010 — logowanie audytowe obejmuje kontekst tenanta.
- architecture.md — warstwa trwała.

# ADR-003: Argon2id + RS256 JWT (asymmetric)

## Status

Accepted

## Kontekst

Platforma przetwarza zastrzeżony kod i wymaga solidnego uwierzytelniania: odporne na ataki brute-force hashowanie haseł, rotacja kluczy oraz możliwość weryfikacji tokenu w wielu serwisach bez udostępniania klucza prywatnego. Tokeny przesyłane w ciasteczkach są podatne na kradzież przez XSS, stąd wymóg HttpOnly + podpisywanie asymetryczne.

## Decyzja

- **Hashowanie haseł:** Argon2id (memory-hard, parametry zgodne z OWASP).
- **Tokeny sesji:** JWT podpisywane **RS256** (asymetrycznie). Klucz prywatny trzymany po stronie backendu (`keys/private.pem`), klucz publiczny (`keys/public.pem`) udostępniany każdemu serwisowi weryfikującemu.
- **Transport:** JWT w **HttpOnly cookie** (`access_token`, `refresh_token`), z flagami `Secure`, `SameSite`. Endpointy akceptują również nagłówek `Authorization: Bearer` dla klientów nieprzeglądarkowych (CLI, SDK).
- **MFA:** opcjonalne TOTP (Fernet jako szyfrowanie sekretów MFA, `ENCRYPTION_KEY`).

## Konsekwencje

**Pozytywne:**

- Asymetryczne JWT: serwery weryfikują token kluczem publicznym, bez dostępu do prywatnego — mniejsza powierzchnia ataku.
- HttpOnly cookie chroni token przed kradzieżą przez XSS.
- Argon2id uodparnia hasła na ataki GPU/ASIC.
- Rotacja kluczy możliwa bez wymiany sekretu we wszystkich serwisach.

**Negatywne:**

- Wymaga zarządzania parą kluczy (generacja, przechowywanie, rotacja).
- MFA zwiększa złożoność przepływu logowania.
- RS256 jest wolniejszy w podpisywaniu niż HS256 (akceptowalne dla sesji).

## Powiązane

- environment-setup.md — zmienne `JWT_*`, `ENCRYPTION_KEY`.

- api/examples/auth_examples.md — przykłady logowania, refresh, MFA.

# ADR-004: Celery + RabbitMQ (przetwarzanie asynchroniczne)

## Status

Accepted

## Kontekst

Analiza kodu (AST, drzewa zależności) i analizy dokumentacji/UML/GUI są wieloetapowe i czasochłonne (parsowanie, preprocessing, ekstrakcja). Wykonywanie ich synchronicznie w żądaniu HTTP blokowałoby worker uvicorn i przekraczało limity czasowe klienta. Potrzebny jest niezawodny broker zadań z kolejkami tematycznymi, ponawianiem i możliwością monitorowania.

## Decyzja

- **Broker zadań:** Celery z **RabbitMQ** jako brokerem oraz **Redisem** jako result backend (`CELERY_RESULT_BACKEND=redis://...`).
- **Kolejki tematyczne:** `default`, `analysis`, `export`, `security` (worker uruchamiany z `-Q default,analysis,export,security`).
- **Pipeline per moduł analizy:** każda analiza (np. `parse_document_task`) to zadanie Celery; wyniki etapów utrwalane są niezależnie w `AnalysisResult.result_data` (JSONB), dzięki czemu częściowe wyniki są dostępne jeszcze przed zakończeniem całego pipeline'u.
- **Limity:** `CELERY_SOFT_TIME_LIMIT=300`, `CELERY_HARD_TIME_LIMIT=330`, `CELERY_MAX_RETRIES=3` z rosnącymi opóźnieniami.
- **Monitorowanie:** Flower (UI tasków Celery, port 5555) — wdrożony jako **usługa produkcyjna** w `docker-compose.yml` (obraz `mher/flower:2.0`, `depends_on: rabbitmq`, healthcheck TCP), chroniona basic-auth przez `FLOWER_USER` / `FLOWER_PASSWORD`. Lokalnie dostępna też przez `make celery-flower`. UWAGA: Flower wystawia argumenty zadań — bez basic-auth (fallback `flower_dev_password`) nie uruchamiać na serwerze dostępnym w sieci.

## Konsekwencje

**Pozytywne:**

- Endpointy uploadu zwracają natychmiast `202` + `task_id`; klient odpytuje status — żaden request HTTP nie czeka na pełną analizę.
- Błąd jednego modułu nie blokuje pozostałych (non-fatal); parsowanie/preprocessing jako jedyne oznaczają plik `failed`.
- RabbitMQ to dojrzały standard; dobre wsparcie w ekosystemie Python.

**Negatywne:**

- Większa powierzchnia operacyjna: dodatkowe serwisy (rabbitmq, redis, celery-worker, flower) w `docker-compose`; Flower wymaga basic-auth i otwarcia portu 5555 na firewallu.
- Debugowanie zadań asynchronicznych jest trudniejsze niż kodu synchronicznego.
- Wymaga zarządzania idempotentnością zadań (reparse, ponowienia).

## Powiązane

- ADR-001 — celery-worker w tym samym obrazie co backend (`target: production`).
- architecture.md — diagram przepływu asynchronicznego.
- environment-setup.md — zmienne `CELERY_*`, `RABBITMQ_*`.

# ADR-005: tree-sitter jako parser AST

## Status

Accepted

## Kontekst

Platforma analizuje kod źródłowy w wielu językach o różnej jakości i dokumentacji. Potrzebne jest precyzyjne, deterministyczne i wielojęzyczne wykrywanie struktury (funkcje, klasy, importy, parametry) jako podłoże dla generowania przypadków testowych. Parsowanie oparte na wyrażeniach regularnych jest kruche; rozwiązania per-język mnożą koszty utrzymania.

## Decyzja

Wykorzystać **tree-sitter** (biblioteka C z wiązaniami Python) jako jedyny parser AST, z per-language grammar. Obsługiwane języki: Python, Java, C#, C/C++, JavaScript, TypeScript, Go. Wynik analizy (`logic_structures`) — funkcje, klasy, importy, parametry, typy zwracane — utrwalany w `AnalysisResult.result_data`.

## Konsekwencje

**Pozytywne:**

- Precyzyjne, deterministyczne i powtarzalne wykrywanie struktury — te same wyniki między uruchomieniami.
- Wysoka wydajność (parser C) i szerokie wsparcie języków.
- Jednolity model ekstrakcji dla wszystkich języków.

**Negatywne:**

- Wymaga grammar per język; dodanie nowego języka to praca integracyjna.
- Inkrementalne aktualizacje grammar zależą od upstream tree-sitter.
- Wykrywanie semantyki wykraczające poza składnię wymaga warstwy reguł powyżej AST.

## Powiązane

- ADR-006 — generowanie kodu testowego na podstawie struktur z AST.
- ai/ast-analysis.md — opis analizy AST.

# ADR-006: Jinja2 jako silnik szablonów kodu testowego

## Status

Accepted

## Kontekst

Generowany kod testów musi być spójny stylistycznie, zawierać poprawne importy, wzorce asercji i konwencje danego frameworka. Bez ujednoliconego podejścia wygenerowane testy są niespójne i często niegotowe do uruchomienia.

## Decyzja

Generowanie kodu testowego oparte na **Jinja2** — szablony per framework testowy (pytest, JUnit, NUnit, Mocha, Jest oraz frameworki E2E — patrz ADR-007). Szablony przyjmują zmienne z analizy AST (nazwa funkcji, parametry, typ zwracany) i emitują gotowy do uruchomienia kod. Rejestr szablonów pozwala dodawać nowe frameworki bez zmian w rdzeniu generacji.

## Konsekwencje

**Pozytywne:**

- Spójny styl kodu i gotowe-do-uruchomienia testy.
- Łatwe dodawanie nowych frameworków (nowy szablon + rejestracja).
- Szablony dostosowywalne do konwencji projektu.

**Negatywne:**

- Koszt utrzymania szablonów przy zmianach frameworków.
- Ewolucja szablonów wymaga wersjonowania i testów regresyjnych (golden data).

## Powiązane

- ADR-005 — AST dostarcza zmiennych do szablonów.
- ADR-008 — golden data waliduje stabilność szablonów.
- ai/test-code-generation.md — opis generowania kodu per framework.

# ADR-007: Wsparcie wieloframeworkowe dla testów E2E

## Status

Accepted

## Kontekst

Nowoczesne aplikacje webowe i mobilne wymagają testów E2E w różnych przeglądarkach i na różnych platformach. Zespoły używają różnych narzędzi w zależności od preferencji i stacku; ograniczenie się do jednego frameworka zawęża użyteczność platformy. Sekwencje kroków (akcje, selektory, asercje) są jednak wspólne niezależnie od narzędzia.

## Decyzja

Generatory kodu testowego dla trzech głównych frameworków E2E, zbudowane nad silnikiem Jinja2 (ADR-006):

- **Selenium** — klasyczna automatyzacja webowa,
- **Playwright** — nowoczesne podejście async/await z auto-waiting,
- **Appium** — testowanie mobilne (iOS/Android).

Wspólny model specyfikacji testu (kroki `action` / `selector` / `value` + asercje) mapowany na kod danego frameworka.

## Konsekwencje

**Pozytywne:**

- Użytkownik wybiera optymalne narzędzie dla swojego przypadku użycia.
- Wspólny generator testów regresyjnych działa dla wszystkich frameworków.
- Reuse modelu specyfikacji między frameworkami.

**Negatywne:**

- Większa ilość szablonów do utrzymania.
- Konieczność nadążania za aktualizacjami frameworków (Selenium/Playwright/Appium).

## Powiązane

- ADR-006 — silnik szablonów.
- ai/test-code-generation.md — generowanie kodu E2E.

# ADR-008: Golden data do walidacji analiz

## Status

Accepted

## Kontekst

Wyniki analiz (klasyfikacja dokumentu, ekstrakcja wymagań, detekcja niespójności, generowanie testów) mogą się zmieniać między uruchomieniami w wyniku refaktoru — niekoniecznie celowo. Potrzebny jest mechanizm wykrywania regresji: zamrożone, ręcznie przygotowane wyniki odniesienia, przeciw którym wyjście bieżącego kodu jest porównywane.

## Decyzja

System **golden data** — zbiory dokumentów/kodu z ręcznie przygotowanym ground truth. Wykonanie analizy porównuje rzeczywiste wyjście z zamrożonym, a testy bramkowe (`test_doc_golden_metrics`, `docs_regression_checker.check_regression`) failują pipeline przy regresji. Metryki baseline (accuracy, precision, recall) utrzymywane per moduł z jawnymi progami.

Ground truth łączy zamrożone wyjście serwisów (liczności, teksty) z intencją autorską (np. `has_intentional_discrepancies`). Zbiory: `backend/tests/golden_data/` (moduł dokumentacji: `docs/`).

## Konsekwencje

**Pozytywne:**

- Wykrywanie subtelnych regresji zachowania analizy.
- Ślad audytu oczekiwanego zachowania; wsparcie testów regresyjnych.
- Ustalony baseline jakości dla ścieżki natywnej.

**Negatywne:**

- Zarządzanie cyklem życia danych odniesienia (regeneracja przy celowej zmianie).
- Dodatkowa pojemność dyskowa dla snapshotów.
- Wymaga dyscypliny: celowa zmiana zachowania = aktualizacja ground truth.

## Powiązane

- ADR-006 — golden data waliduje stabilność szablonów.
- ai/benchmarks.md — metryki baseline per moduł.

# ADR-009: Przepływ przeglądu wygenerowanych elementów

## Status

Accepted

## Kontekst

Wygenerowane przypadki testowe (oraz wyekstrahowane wymagania, glosariusz, niespójności) mogą nie idealnie odpowiadać standardom projektu. Ręczny przegląd i edycja są niezbędne przed użyciem wyników. Bez jawnych statusów i śledzenia decyzji trudno o kontrolę jakości i audyt.

## Decyzja

Jawny **workflow statusów** dla wygenerowanych/ekstrahowanych elementów, z edycją inline oraz operacjami masowymi:

- **Przypadki testowe (kod):** `draft` → `ready` (po akceptacji) → `passed` / `failed` (po uruchomieniu).
- **Wymagania (dokumentacja):** `extracted` → `accepted` / `rejected` / `edited`.
- **Niespójności:** `open` → `accepted` / `false_positive` / `fixed`.
- **Glosariusz:** `is_verified` + `confirmed_mapping_ids` (potwierdzenie mapowań na kod).

Mutacje statusów utrwalane w `AnalysisResult.result_data` (JSONB); operacje masowe (bulk-approve/reject) dla efektywności. Edycja tekstu oznacza odpowiedni status (`edited`).

## Konsekwencje

**Pozytywne:**

- Kontrola jakości wygenerowanych elementów przed użyciem.
- Pełny ślad decyzji (kto zaakceptował/odrzucił/edytował i dlaczego).
- Operacje masowe zwiększają przepustowość przeglądu.

**Negatywne:**

- Dodatkowy krok w pipeline (przegląd przed gotowością).
- Wymaga zaangażowania recenzentów.
- Może spowolnić pierwsze wdrożenie wyników.

## Powiązane

- ADR-010 — decyzje przeglądu są logowane audytowo.

- api/examples/test_cases_examples.md — endpointy akceptacji/odrzucania.

# ADR-010: Logowanie audytowe operacji

## Status

Accepted

## Kontekst

Platforma przetwarza zastrzeżony kod i dokumentację, a wygenerowane testy mogą zawierać informacje poufne. Wymagana jest rozliczalność wszystkich działań wrażliwych: operacji zapisu, zdarzeń uwierzytelniania, dostępu do danych oraz decyzji w workflow przeglądu (akceptacja/odrzucenie/edycja).

## Decyzja

Kompleksowe **logowanie audytowe** dla operacji zapisu, zdarzeń auth (logowanie, MFA, refresh, blokady po nieudanych próbach) oraz dostępu do danych. Logi zawierają kontekst tenanta i użytkownika (RLS — ADR-002). Endpointy audytowe dostępne dla ról z uprawnieniami ( `/admin/audit-logs` ).

## Konsekwencje

**Pozytywne:**

- Spełnienie wymogów zgodności i możliwość badania incydentów.
- Analityka aktywności użytkowników; diagnozowanie problemów produkcyjnych.
- Ślad decyzji w workflow (ADR-009) powiązany z wykonawcą.

**Negatywne:**

- Dodatkowe wymagania pojemnościowe (polityka retencji logów).
- Może nieznacznie wpłynąć na wydajność zapisu.
- Kwestie prywatności danych w logach (sanityzacja pól wrażliwych).

## Powiązane

- ADR-002 — kontekst tenanta w logach.
- ADR-009 — decyzje przeglądu logowane.
- api/examples/audit_examples.md — endpointy audytowe.

# ADR-011: UML wizualne Tier 2 (SVG/PDF/raster) + adapter visual→structural

## Status

Accepted

## Kontekst

Pierwotnie parser UML przyjmował wyłącznie formaty semantyczne (Tier 1: PlantUML, XMI) — `UMLFormat = {xmi, plantuml}`. Story 3.1b rozszerzyła listę o formaty wizualne: **SVG, PDF, PNG/JPEG** (`{xmi, plantuml, svg, pdf, raster}`), parsowane przez OCR tekstu i wnioskowanie kształtów (`uml/visual/`).

Pojawił się szew między parserem wizualnym a downstream: `build_visual_model` emitował elementy z polami wizualnymi (`element_type`, `bbox`, name-based `source` / `target`), podczas gdy cross-diagram (3.7) i compare-code (3.4) czytały **strukturalne** schema (`element_kind`, `id`, `node_type`, `is_initial` / `is_final`, `source_id` / `target_id`). W efekcie diagramy z OCR po cichu produkowały puste mapowania i puste user flows z `status=COMPLETED` — pipeline raportował sukces, a realnie downstream był pusty (groźniejsze niż jawny stub).

## Decyzja

**Root-fix w producerze** (`build_visual_model`, `uml/visual/shapes.py`, domknięcie 3-1b): parser wizualny emituje **zarówno** pola wizualne, **jak i strukturalne** — klasy → `element_kind` (`class` / `interface` / `enum`) + stabilne `id` (`v{idx}`); węzły activity → `node_type` (`start` / `stop` / `action` / `decision`); węzły state → `is_initial` / `is_final`; relacje → `source_id` / `target_id`. Każdy model Tier 2 oznaczony `source: ocr` + `confidence` (0.0–1.0).

Wybrano root-fix w producerze zamiast izolowanego adaptera w konsumerze, bo compare-code (3.4) i traceability (3.15) czytają elementy **wprost z persystowanego JSONB** — adapter w cross-diagram naprawiłby tylko 3.7. Heurystyka start/stop z topologii DAG + etykiet (OCR nie widzi wypełnionego kółka).

## Konsekwencje

**Pozytywne:**

- Diagramy z OCR realnie zasilać cross-diagram (3.7), compare-code (3.4), traceability (3.15) — spójne schema niezależnie od źródła.
- `ClassMatch.*_match_pct: float | None` — koniec vacuous `100%` przy braku成员; uczciwe `null` sygnalizuje niepełną ekstrakcję.

**Negatywne:**

- OCR inherentnie nie ekstrahuje成员 klasy (atrybuty/metody) ani nie dyskryminuje interface/enum (brak markera wizualnego) — udokumentowane ograniczenie.
- `classify_visual_diagram_type` nigdy nie zwraca `state_machine` (kandydat na osobne domknięcie classifiera).

## Powiązane

- ai/uml-parsing.md — adapter + compare-code.
- ADR-008 — golden SVG weryfikuje ścieżkę wizualną.
- Domknięcie `3-1b` (`_bmad-output/implementation-artifacts/E3/3-1b-domkniecie.md`).

# ADR-012: SikuliX image-based — dedykowany pipeline PNG + `pattern_images`

## Status

Accepted

## Kontekst

SikuliX jest frameworkiem image-based — wygenerowany skrypt referuje elementy przez `Pattern("images/...")`. `PatternCropper` (`crop_element` / `crop_all_components`, Pillow) istniał jako gotowe, przetestowane narzędzie, ale task Celery `_generate_sikulix_code_impl` wołał jedynie generyczne `generate_code_for_test_cases`, ignorując `PatternCropper`. Efekt: skrypt referował `Pattern("images/...")` do plików PNG, które **nigdy nie powstawały** — AC-2 Story 3.13 (bbox crop → PNG patterns) było niespięte w ścieżce generowania.

Brakowało też linkage'u `TestStep` ↔ `GUIComponent` (który komponent/bbox/label należy do którego kroku testu).

## Decyzja

**Osobny pipeline** `sikulix_pipeline.generate_sikulix_code` (domknięcie 3-13), oddzielony od generycznej ścieżki Cypress — SikuliX wymaga obrazów, więc separacja jest uzasadniona warstwowo. Pipeline:

1. **Linkage step→GUIComponent** (`resolve_step_component`): explicit `step.component` z bbox → `component_id` (lookup) → label-match (`target` / `target_element` ↔ `GUIComponent.label`, diakrytyki/whitespace → `_`); brak → krok bez patternu.
2. **Crop**: komponenty pogrupowane po `storage_key` → pobranie screenshotu raz z MinIO → `PatternCropper.crop_all_components` → PNG.
3. **Persystencja**: PNG do MinIO (`{tenant}/{project}/sikuli/{test_case_id}/images/{name}`)
   - nowa kolumna `TestCode.pattern_images` JSONB (`[{name, storage_key, component_id}]`, migracja `026`). Nazwa PNG spójna między cropperem a generatorem.
4. **Graceful degrade**: bez Pillow → `[]` + warning → skrypt ze statycznymi referencjami, `pattern_images = null` (nie crash).

## Konsekwencje

**Pozytywne:**

- Skrypty SikuliX referują realnie wyprodukowane PNG — są uruchamialne.

- `pattern_images` daje mapowanie TestCode→patterny w MinIO (odwraca decision z oryginalnego 3.13 „brak kolumny" — bez niej patterny byłyby osierocone).

**Negatywne:**

- Dwa codegen-pipeline'y (Cypress + SikuliX) zamiast jednego generycznego — uzasadnione różnicą DOM vs image.
- Zależność od Pillow (opcjonalna, image-gated).

## Powiązane

- ai/gui-analysis.md — pipeline PNG + `pattern_images`.
- ai/test-generation-uml-gui.md — codegen.
- ADR-007 — frameworki E2E.
- Domknięcie `3-13` (`_bmad-output/implementation-artifacts/E3/3-13-domkniecie.md`).

# ADR-013: Traceability — semantyka Gherkin-proxy + provenance `source` / `confidence`

## Status

Accepted

## Kontekst

Macierz śledzenia (3.15) łączy elementy źródłowe UML/GUI z testami i pokryciem. Dwa problemy (domknięcie 3-15):

1. **Asymetria tokenów post-3-1b.** `_element_type` czytało `element_type` / `type`, nie `element_kind` — klasa z OCR (`element_type="class"`) → `"class"`, ale ta sama klasa z XMI (`element_kind="class"`) → `"element"`. Po fixie 3-1b (visual emituje `element_kind`) asymetria nie znikała bez zmiany w traceability.
2. **Data-model gap.** Story zakładała linkage element→TestCase→TestCode (`source_flow_id` / `GeneratedTestCode`), które nie istniały. Pokrycie było proxy: COVERED = APPROVED `GherkinFeature` traktowane jako „kod wygenerowany" — semanticznie błędne (Gherkin ≠ kod testowy). `test_case_id` zawsze null.

## Decyzja

1. **`_element_type` czyta `element_kind` pierwsze** (kanoniczny dyskryminator strukturalny), potem `element_type` / `type`; mapa `_UML_TYPE_MAP` uzupełniona o tokeny wizualne (`decision` →activity, `enum` →class; `node` dwuznaczny → parametr `category` z `diagram_type`).
2. **Provenance:** kolumny `source` (`parser` | `ocr`) + `confidence` (FLOAT) na `TraceabilityMatrix` (migracja `025`), eksponowane w `TraceabilityEntryOut`.
3. **Semantyka proxy — uczciwie udokumentowana** (decyzja usera: dokumentuj, nie implementuj nowych relacji): COVERED = APPROVED Gherkin = zaakceptowana specyfikacja BDD, **nie** wygenerowany kod testowy; `test_case_id` null z udokumentowanym powodem (`test_cases` wiąże się z `analysis_results`). Realne linkage element→TestCase→TestCode = follow-up.

## Konsekwencje

**Pozytywne:**

- Spójne typy elementów niezależnie od źródła (OCR vs parser) — koniec asymetrii.
- Provenance pozwala odróżnić elementy z OCR (niższa wierność) w macierzy.
- Szczera semantyka pokrycia (bez udawania, że Gherkin = kod).

**Negatywne:**

- `test_case_id` pozostaje null do czasu realnego linkage'u (follow-up).
- Mapowanie `node` wymaga kontekstu `diagram_type` (parametr `category`).

## Powiązane

- ai/traceability-uml-gui.md — macierz + eksport.
- ADR-011 — `element_kind` z OCR.
- Domknięcie `3-15` (`_bmad-output/implementation-artifacts/E3/3-15-domkniecie.md`).

# ADR-NNN: Tytuł decyzji

## Status

Accepted

## Kontekst

Opis problemu, sił na niego wpływających oraz istotne ograniczenia. Dlaczego decyzja była konieczna w momencie jej podejmowania.

## Decyzja

Podjęta decyzja — konkretne, sprawdzalne stwierdzenie (nie ogólnik). Wskazać wybrane narzędzie / wzorzec / podejście oraz jego kluczowe parametry.

## Konsekwencje

Pozytywne i negatywne skutki decyzji: co zyskujemy, za jaką cenę (koszt utrzymania, złożoność, ograniczenia). Wskazać konsekwencje dla zespołu i kodu.

---

## Powiązane

- Inne ADR, sekcje architektury lub specyfikacji, gdy decyzja ma punkt styku.

# Przykłady API — Upload i analiza kodu (AST)

Routery: `app/api/routes/uploads.py` (przesyłanie kodu) oraz `app/api/routes/analysis.py` (analiza AST, tag `analysis`). Analiza jest asynchroniczna (ADR-004); parser AST to tree-sitter (ADR-005). Opis mechaniki: ai/ast-analysis.md.

## Upload kodu źródłowego

```
curl -X POST "http://localhost:8000/api/v1/projects/{project_id}/upload"
          \
  -H "Cookie: access_token=…" \
  -F "file=@app.zip"
```

`202` + `task_id`. Obsługiwane: pojedyncze pliki oraz archiwa `.zip`. Język wykrywany automatycznie; opcjonalnie można go wymusić.

> Pełna lista obsługiwanych języków i parametrów: `/openapi.json` (tag `uploads`).

## Uruchomienie analizy (async)

```
curl -X POST
          "http://localhost:8000/api/v1/projects/{project_id}/analysis" \
  -H "Cookie: access_token=…" -H "Content-Type: application/json" \
  -d '{}'
```

**202:**

```
{ "task_id": "uuid", "status": "pending", "status_url":
          "/api/v1/tasks/{task_id}" }
```

## Status i wyniki

```
curl "http://localhost:8000/api/v1/tasks/{task_id}" -H "Cookie:
          access_token=…"
curl "http://localhost:8000/api/v1/projects/{project_id}/analysis" -H
          "Cookie: access_token=…"
```

Wynik analizy (`logic_structures`) zawiera wykryte funkcje, klasy, importy, parametry i typy zwracane — utrwalone w `AnalysisResult.result_data` (JSONB).

**Przykładowy fragment wyniku:**

```
{
  "functions": [
    {
      "name": "calculate_total",
      "file_path": "app/cart.py",
      "line_start": 15,
      "line_end": 25,
      "parameters": ["items", "tax_rate"],
      "return_type": "float"
    }
  ],
  "classes": [],
  "imports": []
}
```

## Python (httpx)

```
import httpx
c = httpx.Client(cookies=login_cookies)
c.post(f"http://localhost:8000/api/v1/projects/{pid}/upload",
       files={"file": open("app.zip", "rb")})            # 202
c.post(f"http://localhost:8000/api/v1/projects/{pid}/analysis")  # 202 +
         task_id
# odpytywanie /tasks/{task_id} aż status=completed, potem /analysis
```

## Typowe błędy

| Kod | Przyczyna |
|---|---|
| `202` | Zadanie przyjęte / wynik jeszcze niedostępny (odpytaj `/status`) |
| `404` | Projekt/plik nie istnieje |
| `422` | Nieobsługiwany format archiwum |

# Przykłady API — Logi audytowe (admin)

Logi audytowe są realizowane przez model `AuditLog` (`app/models/audit_log.py`, tabela `audit_logs`) oraz serwis `app/services/admin/audit_service.py` (`log_action`, `list_audit_logs`). Architektura: ADR-010.

## Charakterystyka logów

- **Immutability** — logi nie mają `updated_at`; raz zapisane nie są modyfikowane.
- **Brak FK na `tenant_id`** — logi zachowują `tenant_id` nawet po usunięciu tenanta (zgodność/retencja).
- **Kontekst** — każdy wpis: `action`, `user_id`, `tenant_id`, `created_at` oraz payload szczegółów.

## Co jest logowane

- Zdarzenia auth: rejestracja, weryfikacja email, logowanie, wybór tenanta, refresh, wylogowanie, setup/weryfikacja/usunięcie MFA.
- Operacje wrażliwe: upload plików, uruchomienie analizy, akceptacja/odrzucenie przypadku testowego, usunięcie zasobów.
- Decyzje workflow (ADR-009).

## Dostęp (rola admin)

Wgląd w logi mają role platformowe (Admin). Endpointy audytowe eksponowane są pod tagiem admin — pełna lista parametrów i schemat odpowiedzi w `/openapi.json` (tag `admin`). Zapytanie serwisu obsługuje filtry:

```
from app.services.admin.audit_service import list_audit_logs

# Filtry: tenant_id, action, user_id (każdy opcjonalny)
entries = list_audit_logs(
    session=db_session,
    tenant_id=tenant_uuid,   # RLS: ograniczone do kontekstu
    action="test_case.review",
    user_id=user_uuid,
)
```

## Przykładowy kształt wpisu

```
{
  "id": "uuid",
  "action": "test_case.review",
  "user_id": "uuid-użytkownika",
  "tenant_id": "uuid-tenanta",
  "details": {
    "test_case_id": "uuid",
    "decision": "approve",
    "comment": "Zgodny ze standardem"
  },
  "created_at": "2026-06-23T10:00:00Z"
}
```

## Powiązane

- ADR-002 — kontekst tenanta w logach.
- ADR-009 — logowane decyzje przeglądu.
- `/openapi.json` (tag `admin`) — autorytatywna lista endpointów audytowych.

# Przykłady API — Autentykacja i MFA

Router: `app/api/routes/auth.py` (prefiks `/api/v1/auth`, tag `auth`). Tokeny: JWT RS256 w HttpOnly cookie; nagłówek `Authorization: Bearer` dla klientów nieprzeglądarkowych. Architektura: ADR-003.

## Rejestracja

```
curl -X POST "http://localhost:8000/api/v1/auth/register" \
  -H "Content-Type: application/json" \
  -d '{"email": "user@example.com", "password": "SecurePass123!",
       "full_name": "Jan Kowalski"}'
```

`201` → weryfikacja email; konto aktywne po `/auth/verify-email`.

## Logowanie

```
curl -i -X POST "http://localhost:8000/api/v1/auth/login" \
  -H "Content-Type: application/json" \
  -d '{"email": "user@example.com", "password": "SecurePass123!"}'
```

Sukces (200) ustawia ciasteczka `access_token` / `refresh_token` (HttpOnly). Gdy użytkownik należy do >1 tenantów, zwracana jest lista do wyboru → `/auth/select-tenant`.

**Python (httpx):**

```
import httpx
r = httpx.post(
    "http://localhost:8000/api/v1/auth/login",
    json={"email": "user@example.com", "password": "SecurePass123!"},
)
client = httpx.Client(cookies=r.cookies)  # access_token w sesji
```

**JavaScript (fetch):** `credentials: "include"` przechowuje ciasteczka.

```
await fetch("http://localhost:8000/api/v1/auth/login", {
  method: "POST",
  credentials: "include",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ email: "user@example.com", password:
       "SecurePass123!" }),
});
```

## Wybór tenanta

```
curl -X POST "http://localhost:8000/api/v1/auth/select-tenant" \
  -H "Cookie: access_token=…" -H "Content-Type: application/json" \
  -d '{"tenant_id": "uuid-tenanta"}'
```

Ustawia kontekst RLS dla kolejnych zapytań (ADR-002).

## Odświeżanie i wylogowanie

```
# Refresh — używa refresh_token z cookie
curl -X POST "http://localhost:8000/api/v1/auth/refresh" -H "Cookie:
         access_token=…"

# Logout — unieważnia tokeny
curl -X POST "http://localhost:8000/api/v1/auth/logout" -H "Cookie:
         access_token=…"
```

## MFA (TOTP, opcjonalne)

```
# 1. Setup — zwraca secret + QR
curl -X POST "http://localhost:8000/api/v1/auth/mfa/setup" -H "Cookie:
         access_token=…"

# 2. Confirm — weryfikuje kod z aplikacji TOTP
curl -X POST "http://localhost:8000/api/v1/auth/mfa/confirm" \
  -H "Cookie: access_token=…" -H "Content-Type: application/json" \
  -d '{"code": "123456"}'

# 3. Verify przy logowaniu (gdy MFA aktywne)
curl -X POST "http://localhost:8000/api/v1/auth/mfa/verify" \
  -H "Content-Type: application/json" \
  -d '{"mfa_token": "...", "code": "123456"}'
```

Kody zapasowe: `POST /auth/mfa/backup-codes`. Usunięcie MFA: `DELETE /auth/mfa`.

## Typowe błędy

| Kod | Przyczyna |
|---|---|
| `401` | Nieprawidłowe dane logowania / wygasły token |
| `403` | Konto zablokowane po przekroczeniu limit prób (`MAX_LOGIN_ATTEMPTS`) |
| `422` | Niepoprawne ciało (np. hasło poniżej minimalnej długości) |

# Przykłady API — Analiza dokumentacji (Epic 2)

Router: `app/api/routes/documents.py` (prefiks `/api/v1/documents`, tag `documents`). Autoryzacja JWT/cookie; izolacja RLS per tenant. Cały pipeline natywny (regułowy) — tryb silnika pochodzi z konfiguracji i nie jest parametrem żądania. Architektura: ADR-004, opis pipeline'u: ai/document-analysis.md.

## 1. Upload (async) — przez `/projects/{id}/upload-documents`

```
curl -X POST "http://localhost:8000/api/v1/projects/{project_id}/upload-
        documents" \
  -H "Cookie: access_token=…" \
  -F "file=@specyfikacja_wymagan.pdf" \
  -F "file_category=documentation"
```

**202** — plik do MinIO, rekord `SourceFile`, dispoczowane `parse_document_task`:

```
{ "source_file_id": "a1b2…", "task_id": "e5f6…", "status": "pending",
  "status_url": "/api/v1/tasks/e5f6…" }
```

`document_id` = `source_file_id` w kolejnych wywołaniach.

## 2. Status analizy

```
curl "http://localhost:8000/api/v1/documents/{document_id}/status" -H
        "Cookie: access_token=…"
```

```
{
  "document_id": "a1b2…", "status": "ready", "progress": 80.0,
  "available_results": ["document_preprocessing", "classification",
        "requirements", "glossary"],
  "error": null
}
```

`status`: `uploaded` | `processing` | `ready` | `failed`.

## 3. Pełny wynik analizy

```
curl "http://localhost:8000/api/v1/documents/{document_id}/analysis" -H
        "Cookie: access_token=…"
```

```
{
  "document_id": "a1b2…", "filename": "specyfikacja_wymagan.pdf",
  "classification": { "doc_type": "specyfikacja_wymagań", "confidence":
        100.0 },
  "requirements": [ { "req_id": "WYM-001", "text": "System musi…",
                      "type": "functional", "priority": "critical",
        "status": "extracted" } ],
  "glossary": [ { "term": "MFA", "definition": "…", "term_type":
        "acronym" } ],
  "inconsistencies": [ { "type": "duplicate", "severity": "medium",
        "status": "open" } ],
  "gap_analysis": { "total_requirements": 7, "implemented": 4,
        "coverage_percentage": 57.1 },
  "traceability": { "total_requirements": 7, "full_coverage": 4,
        "coverage_percentage": 57.1 },
  "test_generation": { "total_requirements": 7, "generated_tests": 12 }
}
```

`202` gdy preprocessing jeszcze trwa.

## 4. Wymagania + workflow

```
# Lista
curl "http://localhost:8000/api/v1/documents/{document_id}/requirements"
         -H "Cookie: access_token=…"

# Akceptacja / odrzucenie
curl -X POST
         "http://localhost:8000/api/v1/documents/{document_id}/requiremen
         ts/WYM-001/accept" \
  -H "Cookie: access_token=…"
curl -X POST
         "http://localhost:8000/api/v1/documents/{document_id}/requiremen
         ts/WYM-001/reject" \
  -H "Cookie: access_token=…"

# Edycja
curl -X PUT
         "http://localhost:8000/api/v1/documents/{document_id}/requiremen
         ts/WYM-001" \
  -H "Cookie: access_token=…" -H "Content-Type: application/json" \
  -d '{"text": "System musi umożliwiać logowanie email i hasła oraz
         MFA.", "priority": "critical"}'
```

Status workflow: `extracted` | `accepted` | `rejected` | `edited`.

## 5. Klasyfikacja, sekcje, glosariusz, niespójności

```
curl -X POST
         "http://localhost:8000/api/v1/documents/{document_id}/classify"
         -H "Cookie: access_token=…"
curl
         "http://localhost:8000/api/v1/documents/{document_id}/classifica
         tion" -H "Cookie: access_token=…"
curl "http://localhost:8000/api/v1/documents/{document_id}/sections" -H
         "Cookie: access_token=…"
curl "http://localhost:8000/api/v1/documents/{document_id}/glossary" -H
         "Cookie: access_token=…"
curl
         "http://localhost:8000/api/v1/documents/{document_id}/inconsiste
         ncies" -H "Cookie: access_token=…"
```

Mapowanie glosariusza na kod (Epic 1): `POST /documents/{id}/glossary/map-to-code`. Oznaczanie niespójności: `PUT /documents/{id}/inconsistencies/{inconsistency_id}` (`status`: `open` | `accepted` | `false_positive` | `fixed`).

## 6. Ponowna analiza (reparse)

```
curl -X POST
         "http://localhost:8000/api/v1/documents/{document_id}/reparse" -
         H "Cookie: access_token=…"
```

`202` — ponowne parsowanie/ekstrakcja bez utraty quoty; decyzje workflow zachowane po mapowaniach `req_id` → `status`.

## Moduły zależne (tylko do odczytu)

`gap_analysis`, `traceability`, `test_generation` nie mają osobnych endpointów CRUD — zwracane jako klucze w `GET /documents/{id}/analysis` (pkt 3).

> `/projects/{project_id}/traceability` (Epic 3) to macierz śledzenia źródeł UML/GUI — odrębna funkcjonalność, poza zakresem P1.

## Typowe błędy

| Kod | Przyczyna |
|---|---|
| `202` | Wynik jeszcze niedostępny (preprocessing trwa) — odpytaj `/status` |
| `404` | Dokument / wymaganie / sekcja nie istnieje |
| `422` | Niepoprawne ciało / wartość spoza zbioru (np. status niespójności) |

# Przykłady API — Projekty i upload dokumentacji

Router: `app/api/routes/projects.py` (prefiks `/api/v1/projects`, tag `projects`). Wymaga autoryzacji (cookie/Bearer) i kontekstu tenanta (RLS).

## Utworzenie projektu

```
curl -X POST "http://localhost:8000/api/v1/projects" \
  -H "Cookie: access_token=…" -H "Content-Type: application/json" \
  -d '{
    "name": "Platforma e-commerce",
    "description": "Testy przepływu checkout",
    "repository_url": "https://github.com/org/repo"
  }'
```

**201:**

```
{
  "id": "uuid-projektu",
  "name": "Platforma e-commerce",
  "description": "Testy przepływu checkout",
  "repository_url": "https://github.com/org/repo",
  "status": "created",
  "created_at": "2026-06-23T10:00:00Z"
}
```

## Lista i pobranie

```
curl "http://localhost:8000/api/v1/projects?page=1&limit=20" -H "Cookie:
        access_token=…"
curl "http://localhost:8000/api/v1/projects/{project_id}" -H "Cookie:
        access_token=…"
```

Parametry paginacji: `page` (dom. 1), `limit` (dom. 20).

## Aktualizacja i usunięcie

```
curl -X PUT "http://localhost:8000/api/v1/projects/{project_id}" \
  -H "Cookie: access_token=…" -H "Content-Type: application/json" \
  -d '{"description": "Zaktualizowany opis"}'

curl -X DELETE "http://localhost:8000/api/v1/projects/{project_id}" -H
        "Cookie: access_token=…"
```

`DELETE` → `204`. Usunięcie kaskadowe czyści pliki i wyniki analiz tenanta.

## Preprocessing kodu (async)

```
curl -X POST
          "http://localhost:8000/api/v1/projects/{project_id}/preprocess"
          \
  -H "Cookie: access_token=…"
```

`202` + `task_id`. Raport: `GET /projects/{id}/preprocessing-report`.

## Upload dokumentacji (Epic 2, async)

```
curl -X POST "http://localhost:8000/api/v1/projects/{project_id}/upload-
          documents" \
  -H "Cookie: access_token=…" \
  -F "file=@specyfikacja_wymagan.pdf" \
  -F "file_category=documentation"
```

**202** — plik ląduje w MinIO, tworzony rekord `SourceFile`, dispoczowane zadanie `parse_document_task` (Celery). Kolejne kroki opisane w documents_examples.md.

## Lista plików projektu

```
curl "http://localhost:8000/api/v1/projects/{project_id}/documents" -H
          "Cookie: access_token=…"
```

Zwraca `list[SourceFileResponse]` (status przetwarzania per plik).

## Python (httpx)

```
import httpx
c = httpx.Client(cookies=login_cookies)
proj = c.post("http://localhost:8000/api/v1/projects",
              json={"name": "Platforma e-commerce"}).json()
c.post(f"http://localhost:8000/api/v1/projects/{proj['id']}/upload-
          documents",
       files={"file": open("specyfikacja.pdf", "rb")},
       data={"file_category": "documentation"})  # 202
```

## Typowe błędy

| Kod | Przyczyna |
|---|---|
| `404` | Projekt nie istnieje / nie należy do tenanta |
| `409` | Konflikt (np. duplikat nazwy) |
| `422` | Niepoprawne ciało / nieobsługiwane rozszerzenie pliku |

# Przykłady API — Przypadki testowe i przegląd

Router: `app/api/routes/test_cases.py` (prefiks `/api/v1/test-cases`, tag `test-cases`). Statusy workflow: `draft` → `ready` (akceptacja) → `passed` / `failed` (wykonanie). Architektura przeglądu: ADR-009.

## Utworzenie przypadku testowego

```
curl -X POST "http://localhost:8000/api/v1/test-cases" \
  -H "Cookie: access_token=…" -H "Content-Type: application/json" \
  -d '{
    "project_id": "uuid",
    "title": "Obliczenie sumy koszyka",
    "description": "Suma z pozycjami i podatkiem",
    "type": "unit",
    "priority": "high",
    "test_code": "def test_calculate_total():\n    assert calc([]) == 0"
  }'
```

**201** → `TestCaseResponse` ze statusem `draft`.

## Lista i pobranie

```
curl "http://localhost:8000/api/v1/test-cases?project_id=
        {pid}&status=draft&page=1&limit=20" \
  -H "Cookie: access_token=…"
curl "http://localhost:8000/api/v1/test-cases/{test_case_id}" -H "Cookie:
        access_token=…"
```

Filtry: `status` (`draft` / `ready` / `passed` / `failed`), `type` (`unit` / `integration` / `e2e`), `project_id`.

## Edycja i usunięcie

```
curl -X PUT "http://localhost:8000/api/v1/test-cases/{test_case_id}" \
  -H "Cookie: access_token=…" -H "Content-Type: application/json" \
  -d '{"test_code": "def test_updated():\n    assert True", "is_edited":
        true}'

curl -X DELETE "http://localhost:8000/api/v1/test-cases/{test_case_id}" -
        H "Cookie: access_token=…"
```

`DELETE` → `204`.

## Przegląd (akceptacja)

```
curl -X POST "http://localhost:8000/api/v1/test-
        cases/{test_case_id}/review" \
  -H "Cookie: access_token=…" -H "Content-Type: application/json" \
  -d '{"decision": "approve", "comment": "Zgodny ze standardem"}'
```

`approve` → status `ready`; `reject` → pozostaje `draft` z komentarzem. Decyzja logowana audytowo (ADR-010).

## Wykonanie (async)

```
curl -X POST "http://localhost:8000/api/v1/test-
        cases/{test_case_id}/execute" \
  -H "Cookie: access_token=…"
```

**202** + `task_id`. Historia wykonań:

```
curl "http://localhost:8000/api/v1/test-cases/{test_case_id}/executions"
        -H "Cookie: access_token=…"
```

Zwraca `list[TestCaseExecutionResponse]` (status, czas, logi).

## Python (httpx)

```
import httpx
c = httpx.Client(cookies=login_cookies)
tc = c.post("http://localhost:8000/api/v1/test-cases",
            json={"project_id": pid, "title": "Suma koszyka",
                  "type": "unit", "priority": "high",
                  "test_code": "def t(): assert True"}).json()
c.post(f"http://localhost:8000/api/v1/test-cases/{tc['id']}/review",
       json={"decision": "approve"})
c.post(f"http://localhost:8000/api/v1/test-cases/{tc['id']}/execute")  #
        202
```

## Typowe błędy

| Kod | Przyczyna |
|---|---|
| `404` | Przypadek testowy nie istnieje |
| `409` | Akcja niedozwolona w bieżącym statusie (np. execute dla `draft`) |
| `422` | Niepoprawne ciało / nieznana decyzja review |

# Przykłady API — moduł UML/GUI (Epic 3)

Przykłady wywołań endpointów modułu UML/GUI. Autorytatywna specyfikacja maszynowa: `/openapi.json`, `/docs` (Swagger), `/redoc`. Tagi: `uml`, `gui`, `gherkin`, `codegen`, `test-cases`, `traceability`. Modele: ai/uml-gui-data-models.md.

## Autoryzacja

Wszystkie endpointy wymagają uwierzytelnienia: **Bearer JWT** (`Authorization: Bearer <token>`) lub cookie sesyjne (`/api/v1/auth/login`). Zależność `get_current_user` aktywuje **RLS** (`SET LOCAL app.current_tenant`). Bazowy prefiks: `/api/v1`.

## Kody statusu

| Kod | Znaczenie |
|---|---|
| 200 | OK — zwrócono zasób |
| 202 | Accepted — zadanie asynchroniczne (Celery) przyjęte, zwraca `task_id` |
| 400 | Błąd żądania (puste dane wejściowe, brak kontekstu org) |
| 401/403 | Brak tokena / brak uprawnień (RBAC/RLS) |
| 404 | Zasób nie istnieje |
| 422 | Błąd walidacji (Pydantic/Query) |
| 500 | Błąd serwera |
| 501 | Funkcja niedostępna (np. eksport XLSX/PDF bez biblioteki) |

## UML (Stories 3.1–3.7)

```
# Wgranie diagramu (PlantUML/XMI/SVG/PDF/PNG)
curl -X POST "$API/projects/$PID/uml/upload" \
  -H "Authorization: Bearer $TOKEN" -F "file=@diagram.puml"

# Parsowanie → parsed_uml_models
curl -X POST "$API/projects/$PID/uml/parse" -H "Authorization: Bearer
        $TOKEN"

# Cross-diagram → user flows (asynchronicznie)
curl -X POST "$API/projects/$PID/uml/analyze" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"diagram_ids": null}'
# 202 → {"task_id": "…", "status": "pending"}

# Porównanie diagramu klas z kodem projektu (compare-code)
curl -X POST "$API/projects/$PID/uml/$DID/compare-code" \
  -H "Authorization: Bearer $TOKEN"
```

Lista/zasoby: `GET /projects/{id}/uml`, `GET /projects/{id}/uml/{diagram_id}`, `GET /projects/{id}/uml/flows`, `GET /projects/{id}/uml/flows/{flow_id}`.

## GUI (Stories 3.8–3.10)

```
# Wgranie screenshotu (≤ 20 MB; PNG/JPG/JPEG/BMP/TIFF/WebP)
curl -X POST "$API/projects/$PID/gui/upload" \
  -H "Authorization: Bearer $TOKEN" -F "file=@ekran.png"

# Analiza CV/OCR (asynchronicznie) → gui_components + layout
curl -X POST "$API/projects/$PID/gui/analyze" -H "Authorization: Bearer
        $TOKEN"

# Regresja wizualna (porównanie dwóch screenshotów, Story 3.10)
curl -X POST "$API/projects/$PID/gui/compare" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"baseline_screenshot_id": "…", "comparison_screenshot_id": "…"}'
```

Zasoby: `GET /projects/{id}/gui/{screenshot_id}/result`, `GET/POST /projects/{id}/gui/flows`, `GET /projects/{id}/gui/comparisons`, `GET /projects/{id}/gui/comparisons/{id}/diff`.

## Gherkin / BDD (Story 3.11)

```
# Generowanie scenariuszy z przepływów (asynchronicznie)
curl -X POST "$API/projects/$PID/gherkin/generate" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"flow_ids": ["…"], "feature_name": "Logowanie"}'
# 400, gdy flow_ids puste
```

Zasoby: `GET /projects/{id}/gherkin` (paginacja), `GET /projects/{id}/gherkin/{feature_id}` (`?format=raw` → tekst `.feature`).

## Codegen (Stories 3.12, 3.13)

```
curl -X POST "$API/projects/$PID/codegen/cypress" -H "Authorization:
        Bearer $TOKEN"
curl -X POST "$API/projects/$PID/codegen/sikulix" -H "Authorization:
        Bearer $TOKEN"
curl "$API/projects/$PID/codegen/$CODE_ID" -H "Authorization: Bearer
        $TOKEN"
```

`GET /codegen/{code_id}` zwraca opcjonalne `pattern_images` (`[{name, storage_key, component_id}]`) dla SikuliX — realnie wyprodukowane PNG (zob. ai/gui-analysis.md).

## Traceability (Story 3.15)

```
# Macierz + summary (filtry: source_type, coverage_status, paginacja)
curl "$API/projects/$PID/traceability?
         source_type=uml&page=1&page_size=20" \
  -H "Authorization: Bearer $TOKEN"

# Przebudowa (asynchronicznie)
curl -X POST "$API/projects/$PID/traceability/build" -H "Authorization:
         Bearer $TOKEN"

# Eksport (csv|xlsx|json|pdf)
curl -L "$API/projects/$PID/traceability/export?format=csv" \
  -H "Authorization: Bearer $TOKEN" -o traceability.csv
```

XLSX/PDF funkcjonalne (openpyxl/reportlab w main deps); lazy-import + 501 fallback jako defense-in-depth. Odpowiedź `TraceabilityMatrixOut`: ai/uml-gui-data-models.md.

## Sekwencje wywołań

```
UML:         upload → parse → analyze → gherkin/generate → codegen/cypress
GUI:         gui/upload → gui/analyze → gui/flows → codegen/sikulix
Regresja:    gui/upload (baseline) → gui/upload (comparison) → gui/compare
Traceability: traceability/build → traceability (GET) →
traceability/export
```

## Powiązania ze Stories

| Endpoint(y) | Story |
|---|---|
| `/uml/upload`, `/uml/parse` | 3.1–3.6 |
| `/uml/analyze`, `/uml/flows`, `/uml/{id}/compare-code` | 3.7, 3.4 |
| `/gui/upload`, `/gui/analyze`, `/gui/{id}/result` | 3.8 |
| `/gui/flows`, `/gui/compare`, `/gui/comparisons` | 3.9, 3.10 |
| `/gherkin/*` | 3.11 |
| `/codegen/cypress`, `/codegen/sikulix` | 3.12, 3.13 |
| `/traceability/*` | 3.15 |

# Przegląd API — platforma TestGEN (P1)

Autorytatywna specyfikacja maszyny to **OpenAPI 3.1**, generowana z FastAPI i dostępna na trzy sposoby:

- `GET /openapi.json` — surowy schemat (JSON),
- `/docs` — Swagger UI (interaktywny, „Try it out"),
- `/redoc` — ReDoc (czytelna referencja).

Treść tego katalogu jest **przeglądem i przykładami**; przy rozbieżności zawsze decyduje `/openapi.json`. Endpointy opisane są w języku polskim (`summary`, `description`), terminy techniczne pozostają w oryginale.

## Konwencje

- **Prefiks:** wszystkie ścieżki pod `/api/v1`.
- **Bazowy URL (dev):** `http://localhost:8000`.
- **Content-Type:** `application/json` (poza uploadem — `multipart/form-data`).
- **Identyfikatory:** UUID.

### Uwierzytelnianie

JWT (RS256) w **HttpOnly cookie** (`access_token`, `refresh_token`) z flagami `Secure` / `SameSite`. Klienci nieprzeglądarkowe (CLI, SDK) mogą używać nagłówka `Authorization: Bearer <token>`. Szczegóły: ADR-003, przykłady: auth_examples.md.

### Przetwarzanie asynchroniczne

Operacje czasochłonne (analiza kodu, analiza dokumentacji, wykonywanie testów) zwracają natychmiast **`202 Accepted`** z identyfikatorem zadania:

```
{ "task_id": "uuid", "status": "pending", "status_url":
        "/api/v1/tasks/{task_id}" }
```

Status odpytujemy przez `GET /tasks/{task_id}` lub dedykowany endpoint (`GET /documents/{id}/status`). Architektura asynchroniczna: ADR-004.

### Błędy

Błędy walidacji ciała zwracane są jako **RFC 7807** `application/problem+json` (HTTP `422`). Standardowe kody HTTP: `400`, `401`, `403`, `404`, `409`, `422`, `429`. Przykład:

```
{
  "type": "about:blank",
  "title": "Validation Error",
  "status": 422,
  "detail": "Pole 'name' jest wymagane.",
  "instance": "/api/v1/projects"
}
```

### Multi-tenancy i RLS

Każde zapytanie wykonuje się w kontekście tenanta ustawionym w sesji (`app.current_tenant`); polityki RLS wymuszają izolację danych (ADR-002). Wybór tenanta następuje po logowaniu (`/auth/select-tenant`).

## Mapa domen (P1 = Epic 1 + Epic 2)

| Domena | Router | Przykłady |
|---|---|---|
| Autentykacja i MFA | `auth.py` (`/auth`) | auth_examples.md |
| Projekty, członkowie, upload dokumentacji | `projects.py` (`/projects`) | projects_examples.md |
| Upload i analiza kodu (AST) | `uploads.py`, `analysis.py` | analysis_examples.md |
| Przypadki testowe i przegląd | `test_cases.py` (`/test-cases`) | test_cases_examples.md |
| Analiza dokumentacji (Epic 2) | `documents.py` (`/documents`) | documents_examples.md |
| Logi audytowe (admin) | model `AuditLog` + serwis audytu | audit_examples.md |

Endpointy Epic 3 (UML/GUI, Gherkin, traceability źródeł) i Epic 4 (skany bezpieczeństwa, CVE) są poza zakresem P1 — patrz `/openapi.json`.

# Architektura modułu billingowego — przegląd

> Dokumentacja P2, Epic 7 — moduł billingowy TestGEN. Wersja odpowiada implementacji w `backend/app/services/billing/`.

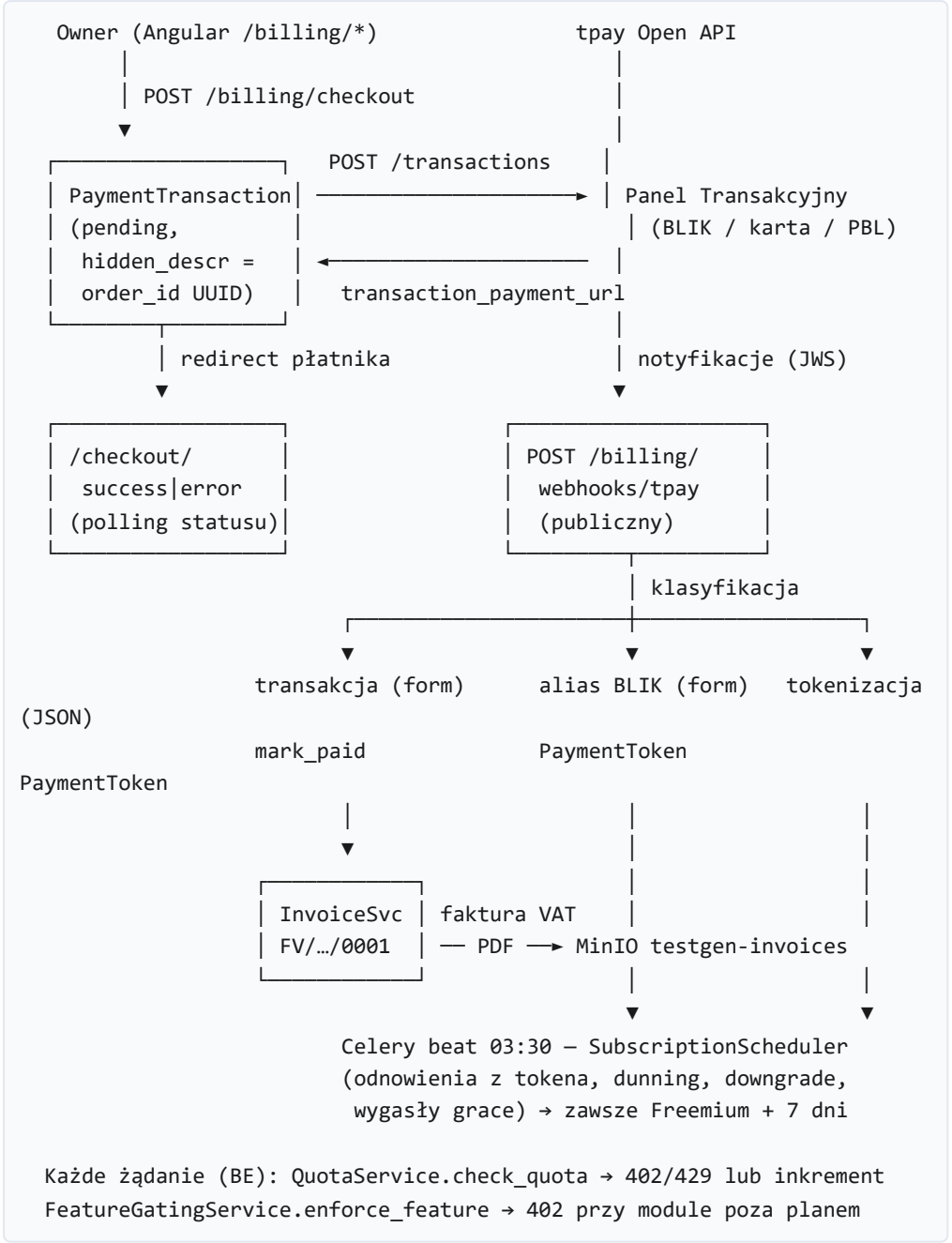
## 1. Cel modułu

Moduł billingowy obsługuje monetyzację platformy: plany subskrypcji, płatności przez tpay, faktury VAT, egzekwowanie limitów per plan oraz kontrolę dostępu do modułów analizy (feature gating).

Główne funkcje:

- **FR54** — wybór planu subskrypcji (Freemium/Basic/Premium)
- **FR55** — zmiana planu w dowolnym momencie (proration, zaplanowane downgrade)
- **FR56** — płatności tpay: BLIK, karta, przelew (change request 2026-08-28 — jedyny provider, zastąpił Stripe + Przelewy24)
- **FR57** — automatyczne faktury VAT po każdej płatności
- **FR58** — egzekwowanie limitów per plan (analizy, test case'y, storage, pliki)
- **FR59** — powiadomienia o zbliżaniu się do limitów (80/90/100%)
- **FR52** — dashboard zużycia zasobów (liczniki, prognoza, historia)
- **FR71** — freemium feature gating — moduły analizy widoczne-zablokowane

## 2. Diagram przepływów




## 3. Moduły i ich odpowiedzialności

| Moduł | Plik | Rola |
|---|---|---|
| Subscription service | `services/billing/subscription_service.py` | Subskrypcje, zmiany planu, proration, tokeny, dunning |
| Quota service | `services/billing/quota_service.py` | Liczniki zużycia, limity, dashboard, prognoza |

| Moduł | Plik | Rola |
|---|---|---|
| Invoice service | `services/billing/invoice_service.py` | Faktury VAT: numeracja, PDF, wysyłka, dane fakturowe |
| Webhook processor | `services/billing/webhook_processor.py` | Klasyfikacja i przetwarzanie notyfikacji tpay |
| Webhook metrics | `services/billing/webhook_metrics.py` | Liczniki Prometheus + snapshot health |
| Subscription scheduler | `services/billing/subscription_scheduler.py` | Zadania okresowe: odnowienia, dunning, downgrade |
| Feature gating | `services/billing/feature_gating.py` | Progi modułów per plan, zależność `require_feature` |
| Provider tpay | `services/billing/providers/tpay/` | Klient Open API: `auth.py`, `client.py`, `transactions.py`, `tokens.py`, `webhooks.py` |
| Notification service | `services/notification_service.py` | Powiadomienia o progach 80/90/100% (in-app + email) |
| Rate limit | `middleware/rate_limit.py` | Limit żądań/min per plan (100/100/1000) |
| Router API | `api/routes/billing.py` | 23 endpointy `/api/v1/billing/*` |
| Router webhooków | `api/routes/webhooks_tpay.py` | `POST /webhooks/tpay` (publiczny) + `GET /webhooks/health` (admin) |
| Seed planów | `seed/seed_plans.py` | Idempotentny upsert planów w startup hooku |
| FE billing | `frontend/src/app/features/billing/` | Portal: plans, checkout, payment-methods, subscription, transactions, invoices, usage |

## 4. Zadania okresowe (Celery beat)

`backend/app/workers/celery_app.py`:

| Zadanie | Harmonogram | Rola |
|---|---|---|
| `billing-usage-snapshot-daily` | 00:00 | Snapshot liczników do `usage_history` (zamyka dzień przed zmianą okresu) |
| `billing-subscription-renewals-daily` | 03:30 | Odnowienia cykliczne z tokenów, dunning, wygasły grace, zaplanowane downgrade |
| `billing-check-thresholds-hourly` | co godzinę | Progi zużycia 80/90/100% → powiadomienia (dedup per okres) |

Kolejność 00:00 → 03:30 jest celowa: snapshot dnia zamyka się przed zmianą okresu rozliczeniowego.

## 5. Konfiguracja

Konfiguracja w `backend/app/core/config_billing.py` (env, bez deployu kodu):

- Ceny: `BILLING_BASIC_MONTHLY_PRICE` (30,00 PLN), `BILLING_PREMIUM_MONTHLY_PRICE` (50,00 PLN), `BILLING_CURRENCY` — Freemium zawsze 0.
- Dokumenty prawne: `BILLING_TERMS_URL`, `BILLING_PRIVACY_POLICY_URL` (regulamin i polityka prywatności osadzane na stronach zakupu — tpay tego wymaga; linki z configu, nigdy hardcode).
- tpay: `TPAY_ENV` (sandbox/production), `TPAY_CLIENT_ID`, `TPAY_CLIENT_SECRET`, `TPAY_SECURITY_CODE`, `TPAY_MERCHANT_ID`, URL-e powrotów i grupy płatności (szczegóły: 03 — Integracja tpay).
- Faktury: dane sprzedawcy `INVOICE_SELLER_*` — snapshot do faktury w chwili wystawienia (szczegóły: 09 — Faktury VAT).

Sekrety tpay wyłącznie z env/secrets — nigdy w kodzie ani repo.

## 6. Zasady projektowe

- **Jedno źródło prawdy limitów** — macierz limitów mieszka w `subscription_plans.features` (JSONB, seed z `seed_plans.py`); cache QuotaService i progi rate limitu zasilane są z bazy przy starcie.
- **Dunning zawsze kończy się Freemium** — nie ma blokady konta za niepłatność; tenant traci plan płatny po 7 dniach grace, dane pozostają.
- **Operacje lokalne vs tpay** — anulowanie/reaktywacja subskrypcji i downgrade są operacjami lokalnymi (bez wywołań API tpay); obciążenia (checkout, odnowienie) zawsze przez tpay.
- **Idempotencja** — webhooki po `event_uid`, faktury po `payment_id`, plany przez upsert w seedzie.
- **RLS** — tabele tenanta (`subscriptions`, `usage_counters`, `invoices`, `payment_transactions`, `payment_tokens`, `plan_change_history`,

`usage_history`, `notifications`) mają polityki `tenant_isolation` + `admin_bypass`; `subscription_plans` to globalna dane referencyjne (bez RLS).

## 7. Powiązane dokumenty

- 02 — Plany i limity
- 03 — Integracja tpay
- 04 — Webhooki
- 10 — Modele danych

# Plany subskrypcji i limity

> Dokumentacja P2, Epic 7 — model 3-planowy (CR#2, 2026-08-28).
> Implementacja: `backend/app/seed/seed_plans.py`, `backend/app/core/config_billing.py`.

## 1. Model 3-planowy

| Plan | Cena miesięczna | Pozycja |
|---|---|---|
| **Freemium** | 0,00 PLN | start — każdy nowy tenant |
| **Basic** | 30,00 PLN (`BILLING_BASIC_MONTHLY_PRICE`) | rozszerzenie analiz |
| **Premium** | 50,00 PLN (`BILLING_PREMIUM_MONTHLY_PRICE`) | pełen dostęp + priorytetowe wsparcie |

Zasady modelu (decyzje CR#2 z 2026-08-28):

- Jedyny cykl: **miesięczny** — brak pól rocznych, brak wersji trial.
- Nowy tenant startuje na Freemium (subskrypcja tworzona automatycznie — `create_freemium_subscription`).
- Dunning (niepłatność odnowienia) **zawsze** kończy się downgrade do Freemium — po 7 dniach grace (szczegóły: 05 — Tokeny i odnowienia).
- Ceny i URL-e regulaminu/polityki prywatności pochodzą z env — wymiana cen = zmiana env + restart (re-seed), bez deployu kodu.

## 2. Macierz limitów

`PLAN_FEATURES` w `seed_plans.py`; `None` = bez limitu:

| Limit | Freemium | Basic | Premium |
|---|---|---|---|
| Analizy / miesiąc | 10 | 50 | ∞ |
| Wygenerowane test case'y / miesiąc | 50 | 250 | ∞ |
| Storage | 0,5 GB | 5 GB | 10 GB |
| Plików na analizę | 5 | 25 | 50 |
| Żądania API / min (rate limit) | 100 | 100 | 1000 |
| Projekty | ∞ | ∞ | ∞ |
| Priorytetowe wsparcie | — | — | ✓ |

Liczniki (`CounterType` w `backend/app/models/usage.py`): `analyses`, `test_cases`, `storage_bytes`, `files_per_analysis`. Egzekwowanie: 07 — Limity i zużycie.

## 3. Progi modułów analizy (feature gating)

Macierz `GATED_MODULES` — minimalny plan odblokowujący moduł:

| Moduł | Minimalny plan |
|---|---|
| Analiza kodu ( `code_analysis` ) | Freemium |
| Generowanie testów ( `test_generation` ) | Freemium |
| Analiza dokumentacji ( `documentation` ) | Basic |
| Analiza UML ( `uml` ) | Basic |
| Analiza GUI ( `gui` ) | Basic |
| Testy integracyjne ( `integration_tests` ) | Basic |
| Generator CI/CD ( `ci_cd` ) | Basic |
| Analiza bezpieczeństwa ( `security` ) | Premium |
| Rekomendacje bezpieczeństwa ( `security_recommendations` ) | Premium |
| Spójność kod↔dokumentacja ( `code_doc_consistency` ) | Premium |
| Luki funkcjonalne ( `functional_gaps` ) | Premium |

Zasada FR71: moduły poza planem są **widoczne-zablokowane** w UI (kłódka + komunikat, który plan odblokowuje), a backend odpowiada **402** — szczegóły: 08 — Powiadomienia i gating.

## 4. Seed planów

`seed_plans.py::seed_subscription_plans` uruchamiany w startup hooku:

1. Upsert 3 planów — brakujące tworzy, istniejące aktualizuje ceną z env (idempotentny; `features` i pola opisowe odświeżane przy każdym starcie).
2. Zasilenie cache limitów QuotaService ( `set_plan_limits_cache` ) — jedno źródło prawdy: baza, nie kod.
3. Zasilenie progów rate limitu ( `set_rate_limits_cache` ) — fallback awaryjny w configu obejmuje scenariusz pustego cache.

Struktura rekordu `subscription_plans` (globalna tabela referencyjna, bez RLS): `name` (unikalny enum), `display_name`, `price_monthly`, `currency`, `features` (JSONB — macierz limitów + `gated_modules` ), `is_active`, `sort_order`.

## 5. Migracja legacy (043)

Migracja `043_migrate_plan_type_to_three_tiers.py` przeprowadziła stare wartości enuma `tenant.plan`: `basic` (dawny darmowy) → `freemium`, `professional` → `premium`. Enum `plan_type_v3` jest źródłem dla hierarchii planów ( `PLAN_HIERARCHY` w `subscription_service.py` : freemium < basic < premium) używanej przy klasyfikacji zmian planu na upgrade/downgrade.

# Integracja tpay — checkout

> Dokumentacja P2, Epic 7 — Stories 7.4, 7.2, 7.3. Implementacja: `backend/app/services/billing/providers/tpay/`, `backend/app/api/routes/billing.py` (POST `/billing/checkout`). Plan integracji: `_bmad-output/planning-artifacts/plan-integracji-tpay.md`.

## 1. Decyzje architektoniczne

- **D1 — jedyny provider**: tpay zastępuje Stripe + Przelewy24 (change request PRD FR56, 2026-08-28). Abstrakcja `PaymentProvider` (`providers/base.py`) pozwala w przyszłości dodać kolejnych dostawców, ale obecnie jedyna implementacja to `TpayProvider`.
- **D2 — minimalna powierzchnia API**: platforma nie robi obciążeń bezpośrednich; każdy przelew pieniędzy odbywa się w Panelu Transakcyjnym tpay (redirect), a stan jest aktualizowany webhookiem. Podgląd proration i operacje anulowania liczone są lokalnie, bez wywołań tpay.
- Sekrety (`TPAY_CLIENT_SECRET`, `TPAY_SECURITY_CODE`) wyłącznie z env/secrets; rotacja + monitorowanie użycia (wymóg tpay).

## 2. Klient Open API

`providers/tpay/`:

| Plik | Odpowiedzialność |
|---|---|
| `auth.py` | Token OAuth (client credentials) dla Open API |
| `client.py` | Wspólny klient HTTP, obsługa błędów `TpayApiError` |
| `transactions.py` | `POST /transactions` (utworzenie transakcji), `GET /transactions/{id}` (reconciliation) |
| `tokens.py` | `POST /tokens` (rejestracja karty), `GET /tokens/{value}` (status), `DELETE /tokens/{value}` |
| `webhooks.py` | Weryfikacja podpisu notyfikacji (JWS + md5sum) — szczegóły: 04 — Webhooki |

Środowiska: `TPAY_ENV=sandbox` → `https://openapi.sandbox.tpay.com`, `production` → `https://api.tpay.com` (nadpisywane przez `TPAY_BASE_URL`).

Grupy płatności Panelu Transakcyjnego: **150 = BLIK**, **103 = karta** (`TPAY_GROUP_ID_BLIK` / `TPAY_GROUP_ID_CARD`). Płatnik wybiera metodę (BLIK, karta z 3DS, przelew/PBL) już w panelu tpay.

## 3. Przepływ checkoutu (Story 7.4)

`POST /api/v1/billing/checkout` — wymaga roli **Owner**:

1. **Guard 409** — tenant z aktywnym planem płatnym nie kupuje ponownie; zmiana planu wyłącznie przez `POST /subscription/change` (Story 7.5).
2. **Rekord lokalny** — `PaymentTransaction` (status `pending`) z `hidden_description` = `order_id` (UUID — webhook koreluje transakcję po polu `tr_crc`) i `meta`: plan, dane płatnika, `blik_alias_value`.
3. **Alias BLIK** — checkout generuje wartość aliasu `tg-{uuid4.hex}` (model A Płatności Powtarzalnych PAYID, Story 7.3 AC3) i przekazuje ją tpay w `blikPaymentData.aliases[0].value`. Notyfikacje `ALIAS_*` dopasowują tenanta **dokładnie** po tej wartości zapisanej w `PaymentTransaction.meta.blik_alias_value`.
4. **Wywołanie tpay** — `POST {TPAY_BASE_URL}/transactions` z kwotą z env, opisem „TestGEN — subskrypcja {Plan} (miesięczna)", danymi płatnika i URL-ami powrotu.
5. **Odpowiedź** — `order_id`, `transaction_id` i `transaction_payment_url`; frontend przekierowuje płatnika na panel tpay.

Limity aliasu BLIK (konfiguracja): płatność pojedyncza = cena planu, limit całkowity = cena × `TPAY_BLIK_ALIAS_TOTAL_LIMIT_PAYMENTS` (12), ważność `TPAY_BLIK_ALIAS_EXPIRATION_DAYS` (730 dni), częstotliwość `eachMonth`.

### Obsługa błędów checkoutu

| Sytuacja | Status | Zachowanie |
|---|---|---|
| Aktywny plan płatny | 409 | Skieruj na `/subscription/change` |
| `refusenopayid` od tpay | 422 | Bank płatnika nie obsługuje Płatności Powtarzalnych BLIK — komunikat z sugestią płatności kartą i zapisania jej jako metody (Story 7.2 AC7) |
| Błąd API tpay / wyjątek | 502 | Rekord pending wycofany (rollback), płatnik może ponowić |

## 4. Strony powrotu i status transakcji

Konfiguracja URL-i (env): `TPAY_SUCCESS_URL` → `/billing/checkout/success`, `TPAY_ERROR_URL` → `/billing/checkout/error`, rejestracja karty: `TPAY_PAYMENT_METHODS_RETURN_URL` → `/billing/payment-methods/return`.

Strony powrotu odpytują `GET /api/v1/billing/transactions/{order_id}` (polling do rozstrzygnięcia statusu). Gdy lokalny status wisi na `pending` dłużej niż próg, endpoint robi **jednorazowy** fallback reconciliation `GET /transactions/{id}` u tpay — okresowy polling transakcji jest wzbroniony wymogami tpay. Rozbieżności między stanem lokalnym a tpay są logowane jako alerty.

Ostatecznym źródłem prawdy o płatności jest **webhook** (aktywacja planu następuje w `mark_paid`, nie na stronie powrotu) — szczegóły: 04 — Webhooki.

## 5. Środowisko sandbox

Runbook: `docs/final/troubleshooting/tpay-sandbox-runbook.md` — konto sandbox, kwoty sterujące scenariuszami płatności (`scripts/set_sandbox_prices.py`), 14 scenariuszy GUI happy-path i błędów. W sandboxu weryfikacja podpisu JWS korzysta z certyfikatu z nagłówka `x5u` — szczegóły w rozdziale webhooków.

# Webhooki tpay

> Dokumentacja P2, Epic 7 — Stories 7.3, 7.4. Implementacja: `backend/app/services/billing/webhook_processor.py`, `backend/app/api/routes/webhooks_tpay.py`. Operacyjny przegląd: `backend/app/services/billing/README.md`.

## 1. Endpoint i autoryzacja

`POST /api/v1/billing/webhooks/tpay` — **publiczny** (bez JWT). Autoryzacją jest podpis notyfikacji:

- **JWS** w nagłówku `X-JWS-Signature` (detached payload) — notyfikacje JSON i form-urlencoded; certyfikat x509 pobierany z `TPAY_JWS_CERT_URL` i cache'owany (TTL 24 h, odświeżenie przy niedopasowaniu).
- **md5sum** — dodatkowo dla notyfikacji transakcyjnych; wzór zaczyna się od `TPAY_MERCHANT_ID` i zawiera `TPAY_SECURITY_CODE`.

tpay nie udostępnia whitelisty IP — podpis jest jedyną linią obrony; wszystkie notyfikacje (także odrzucone) są audytowane w `webhook_events` (commit audytu także przy 400/500).

## 2. Macierz typów notyfikacji

Wszystkie typy trafiają na ten sam URL; klasyfikacja po strukturze (`WebhookProcessor._classify`):

| Typ | Format | Rozpoznanie | Efekt |
|---|---|---|---|
| Transakcja | form-urlencoded | `tr_id` | logika 7.4: `paid` → `mark_paid` (aktywacja/odnowienie planu, faktura), chargeback → ręczna weryfikacja |
| Alias BLIK | form-urlencoded | `event=ALIAS_*` + `msg_value[value]` | aktualizacja `PaymentToken` (status/ważność) po dopasowaniu aliasu |
| Tokenizacja | JSON | `type=tokenization\|token_update`, `data.token` | zapis/aktualizacja tokena karty (serwis 7.2) |
| Inne | dowolny | — | audyt `skipped`, odpowiedź TRUE |

Korelacja alias→tenant: checkout zapisuje wartość aliasu w `PaymentTransaction.meta.blik_alias_value`; notyfikacje `ALIAS_*` dopasowują tenanta po tej wartości (`find_blik_alias_token`).

## 3. Semantyka odpowiedzi

| Wynik | HTTP | Body | tpay |
|---|---|---|---|
| Sukces / pominięcie (idempotencja, nieobsługiwany typ, brak dopasowania) | 200 | `TRUE` (form) / `{"result":true}` (JSON) | przerywa serię ponowień |
| Błąd walidacji (JWS/md5/ śmieciowy payload) | 400 | `FALSE` | **bez ponowień** |
| Wyjątek przetwarzania (np. baza) | 5xx | `FALSE` | ponawia wg harmonogramu |

Harmonogram ponowień tpay: próby 1–10 co 1 min, 11–20 co 3 min, dalej rzadziej — do 37 prób; HTTP 404 przerywa serię. **Idempotencja po** `event_uid` chroni przed podwójnym przetworzeniem tej samej notyfikacji przy ponowieniach.

`event_uid` jest deterministyczny per typ i treść notyfikacji — replika notyfikacji trafia na istniejący rekord `webhook_events` i zwraca sukces bez ponownych efektów ubocznych.

## 4. Efekty transakcji (mark_paid)

`WebhookProcessor.mark_paid` dla transakcji `paid`:

1. `PaymentTransaction.status = paid` (+ `paid_amount`, `test_mode`).
2. Aktywacja planu: pierwszy zakup → `activate_paid_plan`; odnowienie → przesunięcie okresu; upgrade z `meta.plan_change_id` → `finalize_plan_upgrade` (Story 7.5).
3. Faktura VAT: `InvoiceService.generate_invoice` (idempotentna po `payment_id`) → PDF → MinIO → email z załącznikiem (szczegóły: 09 — Faktury VAT).
4. Email potwierdzenia zakupu do płatnika.

Nieudane obciążenie cykliczne (`tr_status` FALSE przy odnowieniu) → `_handle_recurring_declined` → rejestracja porażki w dunningu (szczegóły: 05 — Tokeny i odnowienia). Porównanie `tr_status` jest bezrolkowe (TRUE/FALSE).

## 5. Monitoring (Story 7.3 AC6)

- **Prometheus** (`/metrics`): licznik `webhook_processed_total{provider,type,status}`, histogram `webhook_processing_seconds`.
- **Health panelu admina**: `GET /api/v1/billing/webhooks/health` (rola admin) — liczniki przetworzonych/odrzuconych/pominiętych,

`last_success_age_seconds`, `consecutive_failures`, flaga `alert` (>3 kolejne porażki).

- **Reguła alertu** Prometheus (>3 nieudane przetworzenia w 5 min): konfiguracja w `backend/app/services/billing/README.md`.

Alerty tpay o nieudanych próbach dostarczenia (skrzynka sprzedawcy testgen@nexio.pl po 3 nieudanych próbach) trafiają do właściciela procesu. Procedura reakcji (5 kroków): health → `webhook_events` i logi → obserwacja `consecutive_failures` (tpay ponawia sam) → replay → kontrola rotacji certyfikatów JWS i `TPAY_SECURITY_CODE`.

## 6. Replay (Story 7.3 AC7)

`POST /api/v1/admin/billing/webhooks/{event_uid}/replay` — rola admin, wpis w audycie admina. Ponownie przetwarza zapisany `WebhookEvent.raw` tym samym pipeline'em (walidacja pominięta — dane pochodzą z audytu). Zdarzenia odrzucone walidacją (`rejected_*`) nie podlegają replayowi; idempotencja biznesowa (`event_uid` → `processed`) zapobiega podwójnym efektom. Użycie: rozjazd stanu po dłuższej awarii (np. baza niedostępna w czasie ponowień tpay).

# Tokeny płatności, odnowienia i dunning

> Dokumentacja P2, Epic 7 — Stories 7.2, 7.4. Implementacja: `backend/app/services/billing/subscription_service.py`, `backend/app/services/billing/subscription_scheduler.py`.

## 1. Model PaymentToken

Tabela `payment_tokens` (migracja 045) — zapamiętane metody płatności tenanta, jedna na metodę:

| Pole | Znaczenie |
|---|---|
| `kind` | `card` (token karty) / `blik` (alias BLIK Płatności Powtarzalnych) |
| `value_encrypted` | Wartość tokena zaszyfrowana per-tenant (AES; klucz pochodny z `tenant_id`) |
| `value_digest` | Skrót do wyszukiwania po wartości (indeks) — bez odszyfrowywania |
| `card_tail` / `card_brand` | Wizerunek karty do wyświetlenia (np. `4242` / Visa) |
| `expires_at` | Ważność karty / aliasu |
| `tpay_status` | Status u tpay: `active`, `pending`, `deleted`... (mapowanie `TpayTokenStatus`) |
| `is_default` | Domyślna metoda odnowień |
| `registration_id` | Id rejestracji karty (link do potwierdzenia z notyfikacji) |

Token nigdy nie jest przechowywany jawnie — odszyfrowanie (`decrypt_token_value`) tylko na czas użycia przy obciążeniu.

## 2. Rejestracja karty bez obciążenia (Story 7.2 AC2)

`POST /api/v1/billing/payment-methods` (Owner):

1. Lokalny rekord `PaymentToken` (status `pending`) z `registration_id`.
2. `POST {TPAY_BASE_URL}/tokens` u tpay z `cliAuth` = registration_id — karta zapisywana jest **bez żadnego obciążenia**.
3. Odpowiedź zawiera URL przekierowania na panel tpay; po powrocie płatnika (`/billing/payment-methods/return`, frontend pollinguje potwierdzenie) i notyfikacji tokenizacji (webhook, `data.token`) token przechodzi w `active`.

Listę metod zwraca `GET /billing/payment-methods` — wzbogacana odświeżeniem statusów kart u tpay (`GET /tokens/{value}`) z krótkoterminowym cache'em 300 s; błąd dostawcy → stan lokalny (graceful degrade). Usunięcie metody (`DELETE /billing/payment-methods/{token_id}`) woła `DELETE /tokens/{value}` u tpay i

ustawia lokalnie `tpay_status = deleted` (soft delete). Usunięcie **jedynego aktywnego tokena przy aktywnej subskrypcji cyklicznej** → **409** (najpierw dodaj inną metodę).

## 3. Aliasy BLIK (Płatności Powtarzalne, model A)

Alias powstaje przy pierwszej płatności BLIK: checkout przekazuje tpay wygenerowaną wartość `tg-{uuid4.hex}` (limity: pojedyncza = cena planu, całkowita = cena × 12 płatności, ważność 730 dni, `eachMonth`). Notyfikacje `event=ALIAS_*` (utworzenie/aktywacja/wygaśnięcie/usunięcie) aktualizują `PaymentToken` — korelacja po `meta.blik_alias_value` z transakcji rejestracyjnej. Limit pojedynczy gwarantuje, że odnowienie nie obciąży więcej niż cena planu.

Jeśli bank płatnika nie obsługuje Płatności Powtarzalnych (`refusenopayid`), checkout zwraca 422 z komunikatem — rozwiązaniem jest płatność kartą i zapisanie jej jako metody odnowień.

## 4. Odnowienia cykliczne — scheduler 03:30

`SubscriptionScheduler.run` (Celery beat, codziennie 03:30 — po cleanupie 02:00, przed godziną szczytu; `backend/app/workers/tasks/billing_tasks.py`):

1. **Odnowienia** — subskrypcje z `next_charge_at` ≤ teraz: obciążenie domyślnego tokena (`transactions.py`), nowy okres `current_period_start/end`, `next_charge_at` + 1 miesiąc.
2. **Dunning** — nieudane obciążenie: `register_failed_charge` (inkrement `dunning_attempt`, `dunning_next_retry_at`), `_apply_dunning_downgrade` po wyczerpaniu prób.
3. **Zaplanowane downgrade** — `execute_scheduled_downgrade` dla okresów, które się skończyły (Story 7.5).
4. **Wygasły grace** — `enforce_grace_expiry`: subskrypcje po `grace_until` → ostateczny przebieg do Freemium.

Anulowana subskrypcja (`cancel_at_period_end = true`) nie generuje obciążenia — na koniec okresu następuje powrót do Freemium (lub planu z zaplanowanego downgrade'u, Story 7.5).

## 5. Polityka dunningu

Decyzja modelu (CR#2): dunning **zawsze prowadzi do Freemium** — konto nie zostaje zablokowane, dane pozostają, obowiązują limity Freemium.

```
nieudane obciążenie (odnowienie)
   │  retry wg dunning_next_retry_at (ponowne próby w kolejnych dniach)
   ▼
ostateczna porażka
   │  _apply_dunning_downgrade:
   │    status=active, plan=freemium, previous_plan_id=zapisany,
   │    grace_until = teraz + DUNNING_GRACE_DAYS (7 dni)
   │    email do Ownera: _notify_downgrade (grace 7 dni)
   ▼
grace_until mija → enforce_grace_expiry (idempotentne domknięcie)
```

W ciągu 7 dni grace tenant może dobrowolnie wrócić na plan płatny (zakup/checkout) — limity płatne obowiązują do końca grace. Po wygaśnięciu grace obowiązują limity Freemium; powrót na plan płatny to normalny checkout od zera.

Nieudane obciążenie z aliasu BLIK przy zaplanowanym upgrade (7.5) → `cancel_failed_plan_change`: plan bez zmian, rekord zmiany `failed`, email wyjaśniający.

# Zmiany planu i proration

> Dokumentacja P2, Epic 7 — Story 7.5 (+ interakcje z 7.7). Implementacja: `backend/app/services/billing/subscription_service.py` (`calculate_proration`, `change_plan`, `PlanChangeHistory`).

## 1. Klasyfikacja zmiany

Hierarchia planów: `freemium < basic < premium` (`PLAN_HIERARCHY`).

| Zmiana | Tryb | Rozliczenie |
|---|---|---|
| Upgrade (płatny → wyższy płatny) | płatność natychmiast przez tpay | proration — różnica proporcjonalna do pozostałych dni okresu |
| Z Freemium → Basic/Premium | płatność natychmiast | pełna cena nowego planu (brak komponentu prorated) |
| Downgrade | **lokalnie**, efektywny na koniec okresu | 0,00 — rozliczenie na koniec okresu, bez zwrotów |
| Downgrade → Freemium (anulowanie) | lokalnie (Story 7.7 §3.5) | j.w. — dostęp do końca okresu |

Krawędź: upgrade w **ostatni dzień okresu** daje kwotę 0,00 (wynik ujemny przycięty do zera) — zmiana bez obciążenia, natychmiastowa.

## 2. Formuła proration

Czysta funkcja `calculate_proration` (Decimal, ROUND_HALF_UP, 2 miejsca):

```
płatny → płatny (upgrade):
    kwota = ((cena_nowa / dni_okresu) × pozostałe_dni)
          − ((cena_stara / dni_okresu) × pozostałe_dni)
    kwota  = max(kwota, 0,00)

z Freemium:      kwota = pełna cena nowego planu
downgrade:       kwota = 0,00
```

Wynik zwracany jako słownik audytowy (`amount`, `period_days`, `remaining_days`, ceny obu planów) — trafia do `proration_details` rekordu `PlanChangeHistory`. Podgląd bez skutków: `GET /billing/subscription/proration?plan=…` (Owner; kwota liczona lokalnie — D2, bez API tpay; ten sam plan → 400).

## 3. Przepływ upgrade

`POST /billing/subscription/change` (Owner) → `_start_upgrade`:

1. Blokada subskrypcji (`FOR UPDATE` — `_get_subscription_locked`).

2. `calculate_proration`; kwota > 0 → checkout tpay jak w 7.4 z `meta.plan_change_id`; rekord `PlanChangeHistory` (`pending_payment`).
3. Webhook `paid` → `finalize_plan_upgrade`: natychmiastowa zmiana planu, przesunięcie okresu start na teraz, rekord `completed`.
4. Webhook płatności nieudanej / porażka → `cancel_failed_plan_change`: plan bez zmian, rekord `failed`, email wyjaśniający.
5. Kwota ≤ 0,00 (np. ostatni dnia okresu) → `_apply_free_upgrade`: zmiana natychmiast, bez wywołań tpay.

Ten sam plan → 400; downgrade w trakcie dunningu → 409.

## 4. Przepływ downgrade (zaplanowany)

`_schedule_downgrade`: operacja czysto lokalna — `cancel_at_period_end`, `previous_plan_id`, rekord `PlanChangeHistory` (`scheduled`), zapamiętane `next_charge_at`. Nowy plan obowiązuje od końca bieżącego okresu: `SubscriptionScheduler` (03:30) wykonuje `execute_scheduled_downgrade` po upływie okresu.

Rezygnacja z zaplanowanego downgrade: `POST /billing/subscription/change/cancel-pending` (Owner) — rekord `canceled`, zdjęcie `cancel_at_period_end`, przywrócone `next_charge_at`. Brak oczekującej zmiany → 404.

## 5. Historia zmian

`GET /billing/subscription/history` (Owner) — rekordy `PlanChangeHistory` (migracja 047): typ zmiany (upgrade/downgrade), plan z → plan na, status (`scheduled` / `pending_payment` / `completed` / `failed` / `canceled`), `proration_details`, kwota. Najnowsze pierwsze — audyt pełnego cyklu życia subskrypcji.

## 6. Interakcje z anulowaniem subskrypcji (Story 7.7)

- `cancel_subscription` przy **zaplanowanym downgrade** = operacja idempotentna — rekord zmiany pozostaje nietknięty, na koniec okresu obowiązuje plan niższy (downgrade wykonuje się zamiast powrotu do Freemium).
- `reactivate_subscription` przy oczekującej zmianie planu → **409** (najpierw rozstrzygnij zmianę: `cancel-pending` albo zaczekaj).

# Limity i zużycie zasobów

> Dokumentacja P2, Epic 7 — Stories 7.8, 7.9. Implementacja: `backend/app/services/billing/quota_service.py`, `backend/app/api/routes/billing.py` (`/usage*`), `backend/app/workers/tasks/usage_snapshot.py`.

## 1. Liczniki i egzekwowanie

`QuotaService` operuje na tabeli `usage_counters` (RLS, migracja 042); okno rozliczeniowe = miesiąc kalendarzowy okresu subskrypcji (`_month_window` — od `current_period_start` do `current_period_end`):

| Operacja | Zachowanie |
|---|---|
| `check_quota(tenant, counter)` | `True` / `False` — Czy limit pozwala na +1? |
| `enforce_quota(tenant, counter)` | `check` + rzut 402/429 przy przekroczeniu (używane jako bramka w endpointach) |
| `increment_usage(tenant, counter, value)` | Inkrement w oknie; zapis wartości bezwzględnej dla `storage_bytes` |
| `check_storage_limit(tenant, bytes)` | Osobna ścieżka storage — suma bajtów plików vs limit planu |

Limity pochodzą z `subscription_plans.features` (cache zasilany seedem — jedno źródło prawdy w DB; fallback env tylko przy pustym cache). `None` = bez limitu. Wersje synchroniczne (`increment_usage_sync`, `remaining_quota_sync`) służą procesom workera (sync engine).

Rate limit żądań API (100/100/1000 per min) egzekwuje osobny middleware `app/middleware/rate_limit.py` — progi też z DB przez `set_rate_limits_cache`.

## 2. Dashboard zużycia (Story 7.9)

`GET /billing/usage` — płaskie liczniki; `GET /billing/usage/dashboard` — komplet pod UI (`DashboardData`):

- liczniki per typ z wartością, limitem, procentem zużycia,
- okno rozliczeniowe (start/koniec/dni pozostałe),
- prognoza wyczerpania limitu (`ForecastData`).

Frontend: `/billing/usage` — karty z paskami postępu (`usage-card.component`), wykres historii po rozwinięciu karty (`usage-chart.component` — natywne SVG, bez bibliotek wykresów).

## 3. Prognoza limitu

`calculate_forecast(tenant, counter)`:

- wymaga co najmniej 7 dni danych historycznych (krótszy okres → brak prognozy — brak podstaw statystycznych),
- dzienny przyrost = średnia z `usage_history` w bieżącym oknie,
- ekstrapolacja do końca okresu → flaga `exceeds_before_period_end` (limit zostanie wyczerpany przed końcem okresu rozliczeniowego).

Prognoza pokazywana na karcie licznika jako ostrzeżenie — pozwala z wyprzedzeniem zmienić plan (upgrade natychmiastowy — Story 7.5).

## 4. Historia zużycia

Snapshot liczników codziennie o 00:00 (`billing-usage-snapshot-daily`, `usage_snapshot.py`) do tabeli `usage_history` (RLS, migracja 049). Snapshot zamyka dzień **przed** zmianą okresu (renewals 03:30) — wartość dnia jest stabilna niezależnie od odnowień.

`GET /billing/usage/history?counter=analyses&days=30` — seria punktów `UsageDataPoint` (data, wartość) pod wykres; `GET /billing/usage/history` bez filtra zwraca wszystkie typy.

# Powiadomienia o limitach i feature gating

> Dokumentacja P2, Epic 7 — Stories 7.11, 7.13. Implementacja: `backend/app/services/notification_service.py`, `backend/app/services/billing/feature_gating.py`, `backend/app/api/routes/notifications.py`.

## 1. Powiadomienia o progach zużycia (Story 7.11)

Cykl godzinowy (`billing-check-thresholds-hourly`) → `NotificationService.check_and_notify_thresholds()`:

1. Dla każdego aktywnego tenanta: pobierz zużycie (QuotaService) i Ownera.
2. Próg przekroczony (80 / 90 / 100% któregoś licznika) → `_create_notification`:
   - rekord `notifications` (RLS, migracja 050) — in-app,
   - email (szablon Jinja2 `render_limit_warning_body`) — gdy preferencje kanałów na to pozwalają,
   - **dedup per okres rozliczeniowy** — tenant dostaje każde ostrzeżenie progu (80/90/100%) maksymalnie raz w oknie; kolejne przebiegi godzinowe nie spamują.

Preferencje kanałów (in-app / email) trzyma profil użytkownika — `_get_prefs` / `_prefs_enabled`. Formatowanie wartości per typ licznika (`_format_value`): analizy i test case'y — liczby, storage — MB/GB.

### API i UI

| Element | Lokalizacja |
|---|---|
| `GET /api/v1/notifications` | lista powiadomień z paginacją |
| `GET /api/v1/notifications/unread-count` | licznik nieprzeczytanych |
| `POST /api/v1/notifications/{id}/read` | oznacz jako przeczytane |
| `POST /api/v1/notifications/read-all` | oznacz wszystkie |
| Dzwonek w navbarze | licznik nieprzeczytanych, lista, „oznacz wszystkie" |
| Preferencje w profilu | przełączniki kanałów (in-app / email) |

## 2. Feature gating modułów analizy (Story 7.13, FR71)

Zasada **widoczne-zablokowane**: moduł poza planem jest widoczny w UI z kłódką i komunikatem, który plan go odblokowuje — użytkownik wie, co zyskuje wyższym planem. Blokada nie jest ukryciem.

### Backend

`FeatureGatingService` (`feature_gating.py`):

- `evaluate(tenant, module)` → `(allowed, required_plan)` — próg z `gated_modules` macierzy planów (seed 7.1),
- `enforce_feature(tenant, module)` → rzut **402** z informacją o wymaganym planie,
- `require_feature(module)` — zależność FastAPI do wpięcia w router (używana na 20+ endpointach modułów analizy),
- `GET /api/v1/billing/features` — status wszystkich modułów dla planu tenanta (pod UI: odblokowane/zablokowane + wymagany plan).

### Frontend

`ModuleLocksService` ładuje status modułów i wystawia sygnał reaktywny:

- `locked-feature-tab` / `locked-module` — zakładka/panel z kłódką,
- banner 402 — globalna obsługa odpowiedzi 402 (komunikat + CTA do planów),
- **reaktywny unlock** — po zmianie planu statusy odświeżają się bez przeładowania strony.

Macierz progów: 02 — Plany i limity, sekcja 3.

# Faktury VAT

> Dokumentacja P2, Epic 7 — Story 7.6 (FR57). Implementacja: `backend/app/services/billing/invoice_service.py`, modele `backend/app/models/invoice.py` / `invoice_data.py`.

## 1. Cykl życia faktury

Hook w `WebhookProcessor.mark_paid` (po opłacanej transakcji) → `InvoiceService.generate_invoice`:

1. **Idempotencja po `payment_id`** — faktura dla transakcji już istnieje → zwraca istniejącą (webhooki tpay mogą przyjść wielokrotnie).
2. Snapshoty stron: sprzedawca z env (`INVOICE_SELLER_*`), nabywca z `invoice_data` tenanta (dane fakturowe, Story 7.6 AC4). Snapshot w rekordzie faktury — zmiana env/danych **nie dotyka** już wystawionych faktur.
3. Podział VAT: stawka **23%**, brutto → (netto, VAT) z ROUND_HALF_UP na groszach (`split_vat`).
4. Numeracja (sekcja 2), pozycja faktury z opisem okresu.
5. `render_and_deliver`: PDF (reportlab) → MinIO → email z załącznikiem.

Format kwot: polski separator (`format_pln` — `30,00`), kwoty w rekordzie jako `str` (JSONB nie przechowuje Decimal; precyzja zachowana).

## 2. Numeracja — `FV/{rok}/{miesiąc}/{sekwencja}`

Przykład: `FV/2026/06/0001`. Sekwencja **per miesiąc kalendarzowy**, pole `sequential_number` + unikat `(year, month, sequential_number)`.

Współbieżność (wyścig na pierwszy numer miesiąca — AC6):

- legalny lock numeracji w webhooku (nie transakcyjny SELECT … FOR UPDATE na całości — webhook musi się szybko committing),
- **savepoint + retry** — porażka `unique_violation` cofa tylko tę fakturę (savepoint), nie transakcję webhooka; do 3 prób (`_NUMBERING_ATTEMPTS`).

## 3. PDF, storage, wysyłka

| Krok | Szczegóły |
|---|---|
| Render PDF | reportlab, szablon Jinja2 (`_render_pdf`, `_template_context`) |
| Storage | MinIO, bucket **`testgen-invoices`**, klucz `{tenant_id}/{rok}/{miesiąc:02d}/{numer}.pdf` (`invoice_pdf_object_name`) |
| Wysyłka | Email do Ownera tenanta z załącznikiem PDF (Graph `fileAttachment`) |

| Krok | Szczegóły |
|---|---|
| Błędy wysyłki | Nie wywracają webhooka — faktura istnieje, PDF do pobrania z portalu |

## 4. Dane fakturowe tenanta (AC4)

`invoice_data` (migracja 048): nazwa, NIP, adres — osobno od profilu tenanta:

- `GET /api/v1/billing/invoice-data` (Owner) — brak rekordu → inicjalizacja z profilu tenanta (nazwa, NIP),
- `PUT /api/v1/billing/invoice-data` (Owner) — zmiana dotyczy wyłącznie **następnych** faktur; wystawione trzymają snapshot z chwili wystawienia.

## 5. API faktur

| Endpoint | Rola | Opis |
|---|---|---|
| `GET /api/v1/billing/invoices` | Owner, User | Lista faktur (paginacja 20) |
| `GET /api/v1/billing/invoices/{id}/pdf` | Owner, User | Pobranie PDF (stream z MinIO) |
| `GET /api/v1/billing/invoice-data` | Owner | Dane fakturowe |
| `PUT /api/v1/billing/invoice-data` | Owner | Aktualizacja danych fakturowych |

Frontend: `/billing/invoices` (lista + pobieranie PDF, AC5) i `/billing/invoice-data` (formularz Ownera).

# Modele danych modułu billingowego (ERD)

> Dokumentacja P2, Epic 7. Modele: `backend/app/models/` — `subscription_plan.py`, `subscription.py`, `payment.py`, `plan_change.py`, `invoice.py`, `invoice_data.py`, `usage.py`, `usage_history.py`, `notification.py`.

## 1. Diagram relacji

```
tenants ──1:1── subscriptions ──n:1── subscription_plans (global, bez
RLS)
   │                │
   │                ├──1:n── plan_change_history (audyt zmian)
   │                └──1:1── payment_tokens (is_default → token odnowień)
   │
   ├──1:n── payment_transactions ──1:n── webhook_events (audyt,
event_uid)
   │              │
   │              └──1:1── invoices (idempotencja po payment_id)
   │                              │
   ├──1:1── invoice_data (dane fakturowe → snapshot nabywcy)
   │
   ├──1:n── usage_counters (4 typy liczników, okno miesięczne)
   ├──1:n── usage_history (snapshot dzienny 00:00)
   └──1:n── notifications (progi 80/90/100%, RLS)
```

Wszystkie tabele tenanta mają polityki RLS `tenant_isolation` + `admin_bypass` (wzorzec z migracji 041 — `current_setting('app.current_tenant')`).

## 2. Tabele

### subscription_plans (042 — globalna, bez RLS)

`name` (enum `freemium|basic|premium`, unikalny), `display_name`, `price_monthly` `NUMERIC(10,2)`, `currency`, `features JSONB` (macierz limitów + `gated_modules`), `is_active`, `sort_order`.

### subscriptions (042, 045, 047)

Dokładnie **jedna na tenanta** (`ix_subscriptions_tenant_id` unique): `plan_id`, `status` (`active|past_due|canceled|incomplete`), `current_period_start/end`, `next_charge_at` (indeks — scheduler), `cancel_at_period_end`, `trial_end`, `payment_token_id` (metoda odnowień), `dunning_attempt`, `dunning_next_retry_at` (045), `grace_until` (045), `previous_plan_id` (047 — plan przed dunningiem).

### payment_transactions (044)

`provider` ("tpay"), `transaction_id` (ID u tpay), `title`, `hidden_description` (order_id UUID — korelacja webhooka po `tr_crc`, unikalny, indeks), `amount NUMERIC(10,2)`, `currency`, `status` (`pending|paid|failed|…`), `paid_amount`, `test_mode`, `meta JSONB` (plan, payer, `blik_alias_value`, `plan_change_id`).

### webhook_events (044, 046)

Audyt każdej notyfikacji, także odrzuconej: `event_uid` (unikalny — idempotencja), `type` (transaction/alias/tokenization), `raw JSONB`, `processing_status` (`processed|skipped|error|rejected_*`), `error_message` (046).

### payment_tokens (045)

Metody płatności: `kind` (`card|blik`), `value_encrypted` (AES per-tenant), `value_digest` (indeks wyszukiwania), `card_tail`, `card_brand`, `expires_at`, `tpay_status` (`TpayTokenStatus`), `is_default`, `registration_id` (unikalny — link do potwierdzenia rejestracji karty).

### plan_change_history (047)

Audyt zmian planu: typ (`upgrade|downgrade`), plan z/na, status (`scheduled|pending_payment|completed|failed|canceled`), `proration_details JSONB`, kwota.

### invoices + invoice_data (048)

Faktura: `invoice_number` (`FV/{rok}/{miesiąc}/{seq}`), `year`, `month`, `sequential_number` (unikat razem z year/month), `issue_date`, `sale_date`, snapshoty sprzedawcy i nabywcy (`seller_name/nip/address`, `buyer_name/nip/address`), `items JSONB`, `subtotal_net`, `vat_rate`, `vat_amount`, `total_gross`, `currency`, `payment_method`, `payment_id` (idempotencja), `status`.

Dane fakturowe: 1:1 z tenantem — nazwa, NIP, adres.

### usage_counters (042)

Licznik per tenant i typ (`analyses|test_cases|storage_bytes|files_per_analysis`) w oknie miesięcznym; inkrementy z endpointów i zadań.

### usage_history (049)

Snapshot dzienny (00:00): data, typ licznika, wartość — pod wykresy i prognozy (≥7 dni danych).

### notifications (050)

Powiadomienia in-app: `user_id`, typ (próg zużycia), treść, `read_at`, `dedup_key` (per tenant/okres/próg).

## 3. Rejestr migracji

| Migracja | Zakres | Story |
|---|---|---|
| 042 | `subscription_plans`, `subscriptions`, `usage_counters` + RLS | 7.1 |
| 043 | migracja danych do modelu 3-planowego (`plan_type_v3`) | 7.1 (CR#2) |
| 044 | `payment_transactions`, `webhook_events` | 7.4 |
| 045 | `payment_tokens`, pola dunningu subskrypcji | 7.2 |
| 046 | `webhook_events.error_message` | 7.3 |
| 047 | `plan_change_history` (renumber po kolizji z 7.5) | 7.5 |
| 048 | `invoices`, `invoice_data` | 7.6 |
| 049 | `usage_history` | 7.9 |
| 050 | `notifications` | 7.11 |

Konwencja migracji: idempotentny raw SQL, `DO $$ … EXCEPTION` na dublowane enumy, advisory lock w `migrations/env.py` (kolizje revision ID przy równoległych story — patrz `_ENUMS` w 042).

# Architektura frontendu — przegląd

> Dokumentacja P2, Epic 6 — interfejs webowy TestGEN. Wersja odpowiada implementacji w `frontend/src/app/`.

## 1. Cel

Frontend dostarcza kompletny interfejs platformy: zarządzanie projektami i uploadem, uruchamianie i konfigurację analiz, przeglądanie wyników, zarządzanie test case'ami, raporty i eksporty (FR43–FR46) — w standardzie dostępności **WCAG 2.1 AA** na każdym ekranie.

## 2. Stack technologiczny

| Warstwa | Technologia |
|---|---|
| Framework | Angular 21, **komponenty standalone** (bez NgModules) |
| UI kit | Angular Material (MD3) |
| Stan | **Signals** + `rxjs-interop` (`toSignal` / `takeUntilDestroyed`) |
| Style | SCSS, tokeny w `shared/styles/` |
| Testy jednostkowe | Vitest przez `ng test` (builder `@angular/build:unit-test`) — ze ścisłą type-checką speców |
| Testy E2E | Playwright + axe-core (`frontend/e2e/`) |
| Powłoka natywna | Flutter 3.38 WebView (`mobile/`) — patrz 07 — Shell Flutter |
| Język UI | Polski (pełny interfejs, łącznie z komunikatami błędów i paginacją Material) |

Konwencje kodu: serwisy wstrzykiwane przez `inject()`, dane z HTTP przez signals, subskrypcje wiązane z cyklem życia komponentu (`takeUntilDestroyed`), wszystkie stringi UI po polsku.

## 3. Routing

`frontend/src/app/app.routes.ts` — lazy loading komponentów (`loadComponent` / `loadChildren`), przekierowanie główne → `/dashboard`, wildcard `**` → 404. Grupy tras:

| Grupa | Przykłady tras |
|---|---|
| Autoryzacja | `/login`, `/register`, `/verify-email`, `/set-password`, `/reset-password` `tenant` |

| Grupa | Przykłady tras |
|---|---|
| Dashboard | `/dashboard` |
| Projekty | `/projects`, `/projects/:id/{files,preprocessing,documents,analysis` |
| Moduły analiz | `…/{requirements,glossary,inconsistencies,coverage-report,gap-ana errors,recommendations,comparison,integrations,traceability,secu` |
| Generatory kodu | `…/codegen/{cypress,database,php,…}` |
| Test case'y | `…/test-cases`, `…/test-cases/:testCaseId` |
| Raporty | `…/reports` |
| Ustawienia | `/settings/{profile,mfa,users}`, `/users/invite` |
| Billing | `/billing` (lazy — `BILLING_ROUTES`) |
| Admin | `/admin`, `/admin/cve-pipeline` (lazy) |
| Błędy | `/403`, 404 (wildcard) |

Ochrona tras: `auth.guard.ts` (zalogowany) i `role.guard.ts` (rola — np. `/admin` wymaga Admin, ustawienia użytkowników — Owner).

## 4. Komunikacja z backendem

- HTTP przez `HttpClient`; `auth.interceptor.ts` dokleja token/nagłówki uwierzytelnienia i obsługuje 401; `snake-case.interceptor.ts` normalizuje konwencję nazewniczą pól (backend snake_case ↔ frontend camelCase) — źródło prawdy o kształcie pól w AOT (błędy widać dopiero w `ng build --configuration=production`).
- Konfiguracja runtime: `core/services/config.service.ts`.
- Błędy HTTP mapowane na polskie komunikaty na ekranach.

## 5. Dostępność jako wymiar poprzeczny

WCAG 2.1 AA obowiązuje na **każdym** ekranie — fundamenty (semantyka, fokus, skip-link), motywy (wysoki kontrast, redukcja animacji), ARIA/czytniki ekranu oraz audyt axe-core w testach. Rozdziały 04 — Fundamenty, 05 — Motywy, 06 — ARIA, 08 — Testy.

## 6. Powiązane dokumenty

- `docs/final/deployment-guide/getting-started.md` — uruchomienie frontendu lokalnie.
- `docs/final/user-guide/billing/` — ekrany `/billing/*`.
- Wzorce backendu: `docs/final/architecture/system.md`.

# Struktura aplikacji Angular

> Dokumentacja P2, Epic 6. Katalog: `frontend/src/app/`.

## 1. Warstwy

```
frontend/src/app/
├── app.routes.ts          # routing główny (lazy loading)
├── core/                  # infrastruktura aplikacji (raz, globalnie)
│   ├── guards/            # auth.guard, role.guard
│   ├── interceptors/      # auth.interceptor, snake-case.interceptor
│   └── services/          # auth.service, config.service, module-
locks.service
├── shared/                # elementy wielokrotnego użytku
│   ├── components/        # navbar, footer, file-upload, live-region, …
│   ├── directives/        # dostępność: focus-visible, keyboard-nav, …
│   ├── services/          # theme, upload, file-validation, live-
announcer, …
│   ├── styles/            # tokeny SCSS + motywy dostępności
│   ├── i18n/              # polish-paginator-intl (paginacja Material po
polsku)
│   ├── models/ constants/ utils/ validators/ interceptors/ base/
├── features/              # domeny funkcjonalne (lazy)
│   ├── dashboard/ projects/ results/ test-cases/ reports/
│   ├── settings/ users/ errors/ admin/ billing/ …
└── layouts/
```

Zasada: `core` — pojedyncze instancje infrastruktury; `shared` — bez logiki domenowej; `features` — domeny z własnym routingiem (np. `features/billing/billing.routes.ts`).

## 2. Shared — komponenty

| Komponent | Rola |
|---|---|
| `navbar.component` | główna nawigacja (menu, dzwonek powiadomień, profil) |
| `footer.component` | stopka |
| `file-upload.component` | drag & drop uploadu z walidacją i progresem (6-2) — sekcja 4 |
| `live-region.component` | region ARIA live dla komunikatów asynchronicznych |
| `notification-bell` | dzwonek powiadomień o progach limitów (E7) |
| `accessibility-toolbar` | pasek dostępności na każdej stronie (kontrast, animacje) — 05 — Motywy |

| Komponent | Rola |
|---|---|
| `feature-locked-banner`, `locked-feature-tab`, `locked-module` | blokady modułów poza planem (FR71, widoczne-zablokowane) |
| `quota-exceeded-banner` | banner wyczerpanego limitu (E7) |

## 3. Shared — dyrektywy (dostępność)

| Dyrektywa | Rola |
|---|---|
| `focus-visible.directive` | widoczny fokus klawiaturowy (nie myszowy) |
| `keyboard-nav.directive` | nawigacja klawiaturą po widgetach (strzałki, Home/End) |
| `form-accessibility.directive` | etykiety/opisy pól formularzy dla czytników |
| `heading.directive` | porządek nagłówków (h1–h6 bez przeskoku poziomu) |
| `reduced-motion.directive` | respektowanie redukcji animacji w komponentach |

## 4. Shared — serwisy

| Serwis | Rola |
|---|---|
| `theme.service` | motywy dostępności — sygnały `isHighContrastSignal` / `isReducedMotionSignal`, klasy CSS na `<body>` (05 — Motywy) |
| `upload.service` + `file-validation.service` | upload plików, walidacja rozszerzeń/rozmiaru |
| `live-announcer.service` | komunikaty do czytników ekranu (aria-live) |
| `flutter-bridge.service` | mostek do powłoki natywnej (WebView) — 07 — Shell Flutter |
| `notification.service` | powiadomienia in-app (E7) |

## 5. Style i tokeny

`shared/styles/`: `_colors.scss` (paleta), `_focus.scss` (stany fokusu), `_motion.scss` (animacje + punkty redukcji), `_responsive.scss` (breakpointy), `_skip-link.scss` (skip-link), `_sr-only.scss` (klasa dla czytników), `_touch-target.scss` (minimalne rozmiary celów dotykowych), `themes/_high-contrast.scss` (motyw wysokiego kontrastu).

Tokeny SCSS są jedynym źródłem kolorów/fokusu — komponenty nie definiują własnych wartości kontrastowych.

## 6. Core — guards i interceptors

- `auth.guard` —Chroni trasy za logowaniem (przekierowanie na `/login` z powrotem); `role.guard` — wymóg roli (`Owner`, `Admin`) na trasach ustawień użytkowników, panelu admina, bilingu.
- `auth.interceptor` — uwierzytelnienie żądań + obsługa 401 (wylogowanie do ekranu logowania); `snake-case.interceptor` — normalizacja pól JSON (snake_case ↔ camelCase) przy każdym żądaniu/odpowiedzi.

## 7. Konwencje testowe

Każdy komponent/serwis/guard ma `*.spec.ts` (Vitest przez `ng test`, builder `@angular/build:unit-test` ze ścisłą type-checką — łapie regresje interfejsów TS). E2E w `frontend/e2e/` — 08 — Testy.

# Mapa ekranów i przepływów

> Dokumentacja P2, Epic 6 — Stories 6-1…6-8. Implementacja: `frontend/src/app/features/`.

## 1. Dashboard (Story 6-1)

`/dashboard` (`features/dashboard/`): lista projektów z filtrowaniem, statystyki aktywności tenanta, skróty do ostatnich analiz. Dane z `dashboard.service.ts` (backend `/dashboard`).

## 2. Upload (Story 6-2)

Wspólny komponent `shared/components/file-upload.component.ts` używany przez upload kodu i dokumentacji (`features/projects/document-upload/`):

- **drag & drop** lub wybór plików; wielu plików naraz,
- walidacja po stronie klienta (`file-validation.service`): rozszerzenia (`DEFAULT_DOCUMENT_EXTENSIONS`), rozmiar (`DEFAULT_MAX_FILE_SIZE_BYTES`),
- upload równoległy z **progresem per plik** — statusy `pending → uploading → success | failed | cancelled`,
- podsumowanie po zakończeniu (powodzenia / porażki / pominięte) i komunikaty błędów po polsku.

## 3. Konfiguracja i uruchamianie analiz (Story 6-3)

Widoki projektu: wybór źródeł (kod/dokumentacja/UML/GUI), konfiguracja analizy i uruchomienie (`features/projects/analysis/`), raport preprocessingu (`preprocessing-report/`) przed startem, śledzenie postępu zadania. Wynik analizy ląduje w przeglądarce wyników.

## 4. Przeglądarka wyników (Story 6-4)

`features/results/` + `features/projects/*/view`:

| Komponent | Rola |
|---|---|
| `filter-panel` | filtry wyników (typ, waga, status, moduł) |
| `result-detail` | szczegóły znaleziska z kontekstem |
| `code-highlighter` | podświetlanie składni fragmentów kodu |
| `diff-viewer` | widok różnic (diff) — np. kod ↔ dokumentacja |

Ta sama przeglądarka obsługuje wyniki wszystkich modułów (E2–E5) — wspólne komponenty w `features/results/components/`.

## 5. Zarządzanie test case'ami (Story 6-5)

`features/test-cases/`:

| Element | Rola |
|---|---|
| `test-cases-management` | kompletny widok zarządzania (tabela TC projektu) |
| `test-cases-list` / `test-case-detail` | lista i szczegóły TC |
| `test-case-editor` | edycja kroków/oczekiwanych rezultatów |
| `filter-group-panel` | grupowe filtry |
| `bulk-actions-bar` | operacje masowe (status, eksport, usunięcie) |
| `history-dialog` | historia zmian TC |
| `locator-badge` | informacja o lokatorze (testy UI) |
| `field-mapping` | mapowanie pól przy imporcie/eksportcie |
| `tms-export-dialog` | eksport do TMS (CSV/XML) — FR46 |

## 6. Raporty (Stories 6-6, 6-7 — FR43/FR44/FR45)

`features/reports/`:

| Element | Rola |
|---|---|
| `executive-summary` | raport poziomu zarządczego (podsumowanie, metryki) |
| `technical-details` | raport techniczny (szczegóły znalezisk) |
| `report-preview` | podgląd raportu przed eksportem |
| `export-buttons` + `export-progress-dialog` | eksport **PDF, HTML, CSV, Excel, JSON, YAML** (FR45) z progresem |
| `report.service` / `export.service` / `chart-rendering.service` | dane raportów, eksporty, wykresy |

Raporty dwupoziomowe (FR44): executive summary dla zarządu + szczegóły techniczne dla zespołów QA — ten sam źródłowy zbiór wyników.

## 7. Ekran błędów i statusów

`features/errors/` — `/403` (brak uprawnień) i 404 (wildcard). Blokady modułów planem (FR71) nie są błędem — to komponenty `locked-module` / `locked-feature-tab` z CTA do planów (E7).

## 8. Typowy przepływ użytkownika

```
login → select-tenant → dashboard
  → projects → nowy projekt → upload plików (drag & drop)
  → preprocessing-report → konfiguracja analizy → uruchomienie
  → results (filtry, diff, podświetlanie)
  → test-cases (edycja, operacje masowe, eksport TMS)
  → reports (executive/techniczny → eksport PDF/HTML/…)
```

Każdy krok dostępny z klawiatury i czytnika ekranu (sekcja dostępności), a operacje objęte limitami planu sygnalizowane bannerami (quota/402).

# Fundamenty dostępności — WCAG 2.1 AA

> Dokumentacja P2, Epic 6 — Story 6-9 (fundament, poprzecznie dla całego UI). Implementacja: `frontend/src/app/shared/styles/`, dyrektywy w `frontend/src/app/shared/directives/`, `live-region.component`.

## 1. Zasada

Dostępność jest wymogiem poprzecznym: **każdy ekran** TestGEN spełnia WCAG 2.1 poziom AA. Fundamenty (ten rozdział) + motywy (05) + ARIA (06) + automatyczny audyt axe-core w CI (08) tworzą pełny łańcuch.

## 2. Warstwa stylów (`shared/styles/`)

| Plik | Zakres | Wymóg WCAG |
|---|---|---|
| `_skip-link.scss` | link „Przejdź do treści" jako pierwszy element strony | 2.4.1 (bypass blocks) |
| `_sr-only.scss` | klasa tekstów tylko dla czytników (`visually-hidden`) | 1.3.1 |
| `_focus.scss` | widoczny ring fokusu na interaktywnych elementach | 2.4.7 (focus visible) |
| `_colors.scss` | paleta z kontrastem tekstu ≥ 4,5:1 i dużego tekstu ≥ 3:1 | 1.4.3 |
| `_touch-target.scss` | minimalne 44×44 px cele dotykowe | 2.5.8 |
| `_motion.scss` | animacje z poszanowaniem `prefers-reduced-motion` | 2.3.3 |
| `_responsive.scss` | breakpointy; layout działa od 320 px szerokości | 1.4.10 (reflow) |

## 3. Semantyka HTML

- Poprawna struktura nagłówków — wymuszana dyrektywą `heading.directive` (h1 → h2 → h3 bez przeskakiwania poziomów),
- natywne elementy semantyczne (`nav`, `main`, `table` z `th/scope`, `label` powiązany z polem) zamiast div-ów z rolami,
- `lang="pl"` w dokumencie, tytuły stron przez router (`title:` w definicjach tras) — każda zmiana widoku aktualizuje `document.title`.

## 4. Klawiatura

- Cała aplikacja obsługiwana z klawiatury (tab, enter, escape, spacja); modalne okna Material zamykane Escape i z pułapką fokusu,

- `focus-visible.directive` — fokus podświetlony tylko przy nawigacji klawiaturą (nie przy kliku) — pełny kontrast ringu z `_focus.scss`,
- `keyboard-nav.directive` — wzorce nawigacji strzałkami w listach/ widgetach (np. listy rozwijane, tabele z akcjami) zgodne z ARIA Authoring Practices,
- kolejność fokusu zgodna z logiczną kolejnością wizualną (bez dodatkowych `tabindex` dodatnich).

## 5. Komunikaty asynchroniczne

`live-region.component` + `live-announcer.service` — zmiany stanu (postęp analizy, wynik operacji masowej, błędy walidacji) ogłaszane w regionie aria-live (`polite`), więc czytnik ekranu je przeczyta bez przenoszenia fokusu. Bannery (limit, blokada modułu) też trafiają do regionu live przy pojawieniu się.

## 6. Formularze

`form-accessibility.directive` dba o: etykietę każdego pola (`label for` / `aria-label`), opis błędu powiązany przez `aria-describedby`, `aria-invalid` przy błędzie i komunikat o błędzie tekstowy (nie tylko kolor — zasada 1.4.1).

## 7. Weryfikacja

- Automatyczna: axe-core w E2E na kluczowych trasach (próg: **0** naruszeń critical/serious — 08 — Testy),
- komponentowa: specy a11y przy feature'ach (np. `admin.a11y.spec.ts`, `billing.a11y.spec.ts`),
- ręczna: nawigacja klawiaturą + zoom 200% + 320 px (reflow) — checklist w specyfikacji Story 6-9.

# Motywy dostępności — wysoki kontrast i redukcja animacji

> Dokumentacja P2, Epic 6 — Story 6-10. Implementacja: `shared/services/theme.service.ts`, `shared/components/accessibility-toolbar/`, `shared/styles/themes/_high-contrast.scss`, `_motion.scss`, dyrektywa `reduced-motion.directive.ts`.

## 1. Mechanizm

Motywy to klasy CSS na `<body>` sterowane sygnałami `ThemeService`:

| Motyw | Klasa na `<body>` | Signal | Klucz localStorage |
|---|---|---|---|
| Wysoki kontrast | `high-contrast` | `isHighContrastSignal` | `testgen:a11y:high-contrast` |
| Redukcja animacji | `reduced-motion` | `isReducedMotionSignal` | `testgen:a11y:reduced-motion` |

**Priorytet preferencji**: jawny wybór użytkownika (localStorage) > preferencje systemowe (`prefers-contrast: more`, `prefers-reduced-motion: reduce`) > wyłączone. Zmiana preferencji systemowej w trakcie sesji jest śledzona (`matchMedia … change`) i stosowana, dopóki użytkownik nie dokonał jawnego wyboru. localStorage niedostępny (tryb prywatny) — preferencja żyje w pamięci sesji.

## 2. Pasek dostępności (toolbar)

`accessibility-toolbar.component` — montowany w korzeniu aplikacji, widoczny na **każdej stronie**: rozwijalny pasek z dwoma przełącznikami (wysoki kontrast, wyłączenie animacji). Stan przełączników odzwierciedla sygnały; kliknięcie zapisuje preferencję trwale.

## 3. Tryb wysokiego kontrastu

Motyw `themes/_high-contrast.scss` — paleta czarno-biała z akcentami, kontrast liczony na białym tle:

| Element | Kolor | Kontrast |
|---|---|---|
| Tekst | `#000000` | 21:1 |
| Linki/akcje | `#001a99` | 14,3:1 |
| Błędy | `#740006` | 12,4:1 |

| Element | Kolor | Kontrast |
|---|---|---|
| Sukces | `#0d3d1c` | 12,7:1 |
| Ostrzeżenia | `#4d2e00` | 11,7:1 |
| Nieaktywne | `#595959` | 7:1 |

Progi motywu: tekst ≥ **7:1** (poziom AAA), elementy UI ≥ **4,5:1** — celowo ostrzejsze niż minimum AA, by tryb miał zapas po nałożeniu się komponentów Material. Motyw nadpisuje też stany Material (form fields, przyciski, tabele) i wymusza pełne obramowania tam, gdzie kolor był jedynym wyróżnikiem.

## 4. Redukcja animacji

- `body.reduced-motion` wyłącza animacje przejść/transformat globalnie (`_motion.scss` + arkusz motywu),
- `reduced-motion.directive` stosowana w komponentach z własnymi animacjami (progress, wykresy) — animacja zamieniana na zmianę stanu,
- wejściowa detekcja systemowa (`prefers-reduced-motion`) działa także bez interakcji użytkownika.

## 5. Zoom i responsywność

Layout aplikacji pracuje do **200% zoomu** i **320 px** szerokości bez utraty treści i funkcji (reflow 1.4.10): siatki płynne, tabele z przewijaniem poziomym w kontenerze (nigdy cała strona), breakpointy w `_responsive.scss`.

# ARIA i czytniki ekranu

> Dokumentacja P2, Epic 6 — Story 6-11. Implementacja: dyrektywy `shared/directives/`, `live-region.component`, `live-announcer.service`, `i18n/polish-paginator-intl.ts`.

## 1. Zasady użycia ARIA

- **Najpierw semantyka HTML** — ARIA tylko tam, gdzie HTML nie wystarcza (widgety kompozytowe, live regions, opisy kontekstowe),
- żadnej informacji tylko-w-kolorze — stan komunikowany tekstem/ikoną z etykietą (1.4.1),
- role kompozytowe zgodne z ARIA Authoring Practices (listy wyboru, tabele danych z akcjami, dialogi).

## 2. Nawigacja czytnikiem

| Element | Wsparcie |
|---|---|
| Skip-link | „Przejdź do treści" jako pierwszy fokus strony (2.4.1) |
| Nagłówki | dyrektywa `heading` pilnuje hierarchii h1–h6; każda strona ma dokładnie jedno h1 |
| Landmarki | `nav`, `main`, `footer` — nawigacja po regionach |
| Tytuły stron | `title` per trasa (router) — zmiana czytelna w historii i pasku |
| Formularze | `form-accessibility.directive`: etykiety, `aria-describedby` (błędy/podpowiedzi), `aria-invalid` |
| Tabele danych | `th` + `scope`, podpisy (caption), relacje komórka→nagłówek |

## 3. Komunikaty asynchroniczne (aria-live)

- `live-region.component` — wspólny region live w layoucie; `live-announcer.service` — API do ogłaszania („Analiza zakończona", „Zaznaczono 12 przypadków testowych", błędy operacji),
- priorytet `polite` (nie przerywa czytania), `assertive` tylko dla krytycznych błędów blokujących,
- postęp długich operacji (upload, eksport raportu) ogłaszany punktami kontrolnymi, nie strumieniem komunikatów.

## 4. Interakcje klawiaturowe widgetów

`keyboard-nav.directive` standaryzuje wzorce: listy/rozwijane — strzałki góra/dół + Enter/Escape; tabele z akcjami — strzałki między wierszami; toolbar dostępności — TAB między przełącznikami. Fokus widoczny zawsze (`_focus.scss`

+ `focus-visible.directive`).

## 5. Polonizacja komponentów Material

`i18n/polish-paginator-intl.ts` — paginacja Material („Następna strona", „Pozycje od–do z …") oraz etykiety ikar wskaźników po polsku: czytnik ekranu nie słyszy angielskich etykiet domyślnych. Wszystkie stringi UI (aplikacja + komponenty Material + komunikaty błędów HTTP) są po polsku.

## 6. Weryfikacja z czytnikami

- automatycznie: axe-core (E2E) + reguły ARIA w specach komponentowych,
- ręcznie: NVDA (Windows) na przepływach z 03 — Mapa ekranów — checklist Story 6-11: logowanie, upload, wyniki z filtrami, edycja TC, eksport raportu, przełączniki motywów dostępności.

# Powłoka Flutter WebView (mobile/desktop)

> Dokumentacja P2, Epic 6 — Story 6-12. Implementacja: `mobile/` (projekt `testgen_mobile`), mostek po stronie Angulara: `shared/services/flutter-bridge.service.ts`.

## 1. Koncepcja

Natywna powłoka uruchamia **frontend webowy w WebView** — minimalna logika natywna, cały interfejs i logika po stronie Angulara. Jedna baza funkcjonalna (web) + dystrybucja przez sklepy/systemy (aplikacja natywna).

## 2. Platformy

| Platforma | WebView | Plugin |
|---|---|---|
| iOS 15+ | WKWebView | `webview_flutter` |
| Android | Android WebView | `webview_flutter` |
| Windows | Edge WebView2 | `webview_windows` |
| Linux | fallback: przeglądarka systemowa | `url_launcher` (brak stabilnego pluginu WebKitGTK) |

## 3. Struktura (`mobile/lib/`)

```
main.dart                              # entry point
config.dart                            # URL frontendu (dev/prod)
webview/webview_container.dart         # dobór WebView per platforma +
ekran offline
bridge/js_bridge.dart                  # dwukierunkowy mostek JS ⇔ Dart
deep_link/deep_link_handler.dart       # testgen:// → ścieżka Angulara
notifications/notification_handler.dart  # payload powiadomienia →
nawigacja
widgets/offline_message.dart           # komunikat braku sieci
```

## 4. Mostek JS (`js_bridge.dart` ↔ `flutter-bridge.service.ts`)

Dwukierunkowa komunikacja WebView ↔ Flutter:

- **Angular → natywne**: żądania funkcji powłoki (np. obsługa linków zewnętrznych, udostępnienie plików PDF),
- **natywne → Angular**: zdarzenia systemowe, deep linki, powiadomienia.

`flutter-bridge.service.ts` wykrywa, czy aplikacja działa w powłoce — poza nią funkcje mostka są no-op (czysty web działa bez zmian).

## 5. Deep linki i powiadomienia

- **Deep link** `testgen://ścieżka` → `deep_link_handler` tłumaczy na nawigację routera Angulara (np. link z maila otwiera konkretny projekt),
- **powiadomienia push/lokalne** → `notification_handler` przekazuje payload do Angulara, który nawiguje na właściwy ekran (np. powiadomienie o progu limitu → dashboard zużycia).

## 6. Tryb offline

`webview_container` + `widgets/offline_message.dart` — wykrycie braku sieci pokazuje natywny komunikat; po powrocie połączenia WebView odświeża frontend. Cache'owanie treści świadomie minimalne (dane tenanta nie trafiają do pamięci podręcznej powłoki).

## 7. Konfiguracja i budowa

`config.dart` trzyma URL frontendu osobno dla dev i prod; budowa per platforma standardowymi narzędziami Flutter ( `flutter build apk/ios/ipa/ windows` ). Powłoka nie przechowuje sekretów — uwierzytelnienie wyłącznie przez sesję webową (HttpOnly cookies) w WebView.

# Strategia testowa frontendu

> Dokumentacja P2, Epic 6 — Story 6-13. Implementacja: specy `*.spec.ts` przy komponentach, `frontend/e2e/`, `frontend/playwright.config.ts`.

## 1. Poziomy testów

| Poziom | Narzędzie | Zakres |
|---|---|---|
| Jednostkowe/komponentowe | **Vitest przez `ng test`** (builder `@angular/build:unit-test`) | każdy komponent, serwis, guard, dyrektywa ma `*.spec.ts` |
| E2E | **Playwright** (`frontend/e2e/`) | przepływy przez UI na uruchomionym frontendzie |
| Dostępność | **axe-core** (`@axe-core/playwright`) | audyt WCAG na kluczowych trasach (w E2E) |
| AOT/typy | `ng build --configuration=production` | łapie błędy typów w szablonach niewidoczne dla testów |

Uruchomienie testów: `npm test` (= `ng test`). Uwaga: bezpośrednie `npx vitest` omija inicjalizację TestBed i psuje wszystkie specy — zawsze `ng test`. Builder wykonuje też ścisłą type-checkę speców, więc zmiany interfejsów TS (nowe pola → opcjonalne `?`) wywalują testy, zanim trafią do CI.

## 2. Testy komponentowe (Vitest)

- Setup TestBed per spec, mocki HTTP (`provideHttpClient` z interceptorami testowymi), signals testowane przez `signal()` + `fixture.detectChanges()`,
- a11y komponentowo: dedykowane specy dostępności przy dużych feature'ach (`admin.a11y.spec.ts`, `billing.a11y.spec.ts`) — struktura nagłówków, etykiety, role,
- guards/interceptory testowane na routingach i żądaniach HTTP.

## 3. E2E — Playwright

`frontend/e2e/`:

| Plik | Przepływ |
|---|---|
| `auth.spec.ts` | rejestracja, weryfikacja, logowanie, MFA, wybór tenanta |
| `dashboard.spec.ts` | strona główna, lista projektów |

| Plik | Przepływ |
|---|---|
| `upload.spec.ts` | upload drag & drop, walidacje, błędy |
| `analysis.spec.ts` | konfiguracja i uruchomienie analizy, wyniki |
| `report.spec.ts` | raporty dwupoziomowe, eksporty |
| `accessibility.spec.ts` | audyt axe-core (sekcja 4) |

Page Objecty w `e2e/pages/` (`auth.page`, `dashboard.page`, `upload.page`, `analysis.page`, `report.page`), wspólne fixtury w `e2e/fixtures/` (`test-fixtures.ts` — uwierzytelniona sesja, przygotowany projekt).

## 4. Audyt dostępności w CI (axe-core)

`accessibility.spec.ts` — konwencja ze Story 6-9 (`accessibility/ axe-runner.ts` i `shared/utils/axe-setup`):

- tagi: `wcag2a`, `wcag2aa`, `wcag21a`, `wcag21aa`,
- próg zaliczenia: **0 naruszeń o impact `critical` lub `serious`**,
- skan kluczowych tras (dashboard, projekt, wyniki, TC, raporty, billing, admin) w obu motywach dostępności (standardowy i wysoki kontrast).

## 5. Produkcja (AOT)

`ng build --configuration=production` jest obowiązkowym krokiem przed mergem/CI: kompilacja AOT wyłapuje błędy typów w szablonach (nazwy pól camelCase/snake_case po interceptorze, martwe `@if`, typy `$event`), których `ng test` nie widzi. Regresje AOT łamały już deploy Dockera — dlatego build produkcyjny biega przed publikacją obrazu.